

FLEXI: A high order discontinuous Galerkin framework for hyperbolic–parabolic conservation laws

Nico Krais^{a,*}, Andrea Beck^a, Thomas Bolemann^a, Hannes Frank^a, David Flad^a, Gregor Gassner^b, Florian Hindenlang^c, Malte Hoffmann^a, Thomas Kuhn^a, Matthias Sonntag^a, Claus-Dieter Munz^a

^a Institute of Aerodynamics and Gasdynamics, University of Stuttgart, Germany

^b Mathematical Institute, Center for Data and Simulation Science (CDS), University of Cologne, Germany

^c Max Planck Institute for Plasma Physics, Germany

ARTICLE INFO

Article history:
Available online 28 May 2020

Keywords:
Discontinuous Galerkin
High order
Large eddy simulation
Computational fluid dynamics
Open-source software
Shock capturing

ABSTRACT

High order (HO) schemes are attractive candidates for the numerical solution of multiscale problems occurring in fluid dynamics and related disciplines. Among the HO discretization variants, discontinuous Galerkin schemes offer a collection of advantageous features which have led to a strong increase in interest in them and related formulations in the last decade. The methods have matured sufficiently to be of practical use for a range of problems, for example in direct numerical and large eddy simulation of turbulence. However, in order to take full advantage of the potential benefits of these methods, all steps in the simulation chain must be designed and executed with HO in mind. Especially in this area, many commercially available closed-source solutions fall short. In this work, we therefore present the *FLEXI* framework, a HO consistent, open-source simulation tool chain for solving the compressible Navier–Stokes equations on CPU clusters. We describe the numerical algorithms and implementation details and give an overview of the features and capabilities of all parts of the framework. Beyond these technical details, we also discuss the important but often overlooked issues of code stability, reproducibility and user-friendliness. The benefits gained by developing an open-source framework are discussed, with a particular focus on usability for the open-source community. We close with sample applications that demonstrate the wide range of use cases and the expandability of *FLEXI* and an overview of current and future developments.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

The favorable properties of high order (HO) methods for smooth flow problems have long been recognized. Initially, global spectral methods, where the solution in the computational domain is discretized by a single, global set of basis functions were favored. One of the earliest applications of a spectral Galerkin method to flow problems was published by Silberman in 1954 [1]. Many others followed, in particular in the area of turbulence using Fourier (pseudo)-spectral

* Corresponding author.

E-mail addresses: krais@iag.uni-stuttgart.de (N. Krais), beck@iag.uni-stuttgart.de (A. Beck), thomas.bolemann@daimler.com (T. Bolemann), hannes.frank@stihl.de (H. Frank), david.g.flad@nasa.gov (D. Flad), ggassner@uni-koeln.de (G. Gassner), florian.hindenlang@ipp.mpg.de (F. Hindenlang), hoffmann@iag.uni-stuttgart.de (M. Hoffmann), thomas.kuhn@iag.uni-stuttgart.de (T. Kuhn), matthias.sonntag@zeiss.com (M. Sonntag), munz@iag.uni-stuttgart.de (C.-D. Munz).

methods, see e.g. [2–4]. While these methods can offer unsurpassed accuracy per invested degree of freedom, they are naturally limited by their restriction to the global, i.e. single element, domain discretization. This makes them too inflexible for real word applications, and this shortcoming has sparked a lot of research efforts in *local* high order methods during the last decades. From these efforts, high order variants of essentially every grid-based approximation method to partial differential equations (PDE) have sprung — there exist a plethora of finite difference, finite volume, finite element and other high order formulations nowadays, see e.g. [5–10] and many more. What all these methods have in common is that they surrender the notion of a global solution representation in favor of a local but high order one. This locality entails that the differential operators acting on the solution themselves now can become local, which in turn allows a multi-element domain discretization. Since on a conceptual level, these HO methods differ in the way they define the approximate solution ansatz, the locality of their bases and the mathematical principle that recovers the global approximate solution through connection of the now local PDE approximations also vary. These differences do not only influence the numerical properties of the resulting schemes, but have also far-reaching consequences with regard to practical aspects: Which meshes are suitable for which method? How are the physical properties and limitations reflected in the design of the method? How computationally efficient is the algorithm; can it exploit modern vectorization in CPUs and what are its limiting components? How well can the method be run in parallel on tens or hundreds of thousand of CPUs or on multiple GPUs? In particular the last two issues are of importance when considering practical applications of HO methods beyond canonical test cases, most of which stem from the global method community. The multitude of questions and choices to consider should also make it clear that no “best” HO method exists — instead, it depends on the specific requirements of the targeted applications at hand.

In our case and for the framework presented here, these applications are found in the realm of the unsteady, compressible Navier–Stokes equations and related formulations like the Euler system. The particular focus of interest are non-linear multiscale problems like turbulence and aeroacoustics in the compressible flow regime. For these types of problems, discontinuous Galerkin (DG) methods offer a collection of advantageous features. A number of research groups focus on developing numerical frameworks either solely for or incorporating DG methods, such as *Nektar++* [11], the *ATELES* module of *APES* [12], *PyFR* [13] or *Argo* [14]. DG and related methods are based on the variational form of the equations realized by an L_2 Galerkin projection. The polynomial basis and test functions are both restricted to each element, and thus the projection becomes a purely local operator. Connection to adjacent elements is achieved by the weak imposition of a unique surface flux. Thus, it is convenient to interpret DG as a hybrid of an element-local finite element scheme and a finite volume (FV) discretization. This view on DG methods has direct consequences for the approximation properties, the stability and the data connectivity of these methods. Due to their local high order accuracy, the global method is also high order accurate and has very low numerical dispersion and dissipation errors [15]. This property turns HO methods into an attractive candidate even for under-resolved simulations, where the benefits of fast error convergence are not relevant, see e.g. [16,17] for a discussion. The possibility to introduce upwinding through the surface fluxes provides stability for linear hyperbolic problems, and naturally adds stabilizing dissipation in the non-linear case. The connection to neighboring elements is only through weakly imposed numerical fluxes at the interfaces which avoids strong global coupling and reduces the communication pattern to that of first order finite volume schemes, while achieving arbitrary high order of accuracy through the local ansatz degree.

With a certain target application in mind, choosing the most suitable HO method to solve the governing equations is however only the first choice along the way. Any meaningful application of such methods involves making certain that the whole simulation chain is up to the task. In other words, choosing a HO method entails HO at all stages of the process, starting from the pre-processing steps, in particular the generation of grids suitable for high order (both in terms of surface representations as well as sufficient grid coarseness). Especially for wall-bounded problems in aerodynamics, where the flow in the vicinity of the wall is usually well resolved and the geometries might be complex, accurate representation of boundaries is of great importance [18]. Other issues to consider at pre-processing for example include the availability of high order accurate initial and boundary conditions. Post-processing of the solution also poses challenges. For the typical target application such as highly resolved and time-accurate simulation of turbulence, large amounts of data are generated during the simulation, which must be dumped to disk in a raw format to avoid efficiency losses. This raw data has to be post-processed, manipulated and visualized in a way that is also consistent with the HO approach and is compatible with the usually low order data format used by third party software. If the higher information content of a HO data point is not exploited in post-processing, at least the advantage of reduced storage requirements is lost.

Since a successful HO strategy thus requires a concerted approach across all links of the simulation chain and as suitable third party pre- and post-processing solutions are not available, we designed and developed (and continue to do so) the *FLEXI*¹ framework, which will be discussed and presented in this work. The basic idea behind *FLEXI* is not only to provide a HO DG solver for the compressible Navier–Stokes equations that is well-documented and easy to modify in the hope others may find it useful, but to create a full, HO consistent simulation tool chain with a focus on current CPU based cluster systems for high performance computing (HPC). Thus, all parts of *FLEXI* are designed to deal with HO data, and to make use of a large number of cores. In order to make our work available to the scientific community and to facilitate peer-review and reproducibility, *FLEXI* has been made open-source under the GNU GPL v3.0 license. We will present *FLEXI*, its features, parts, numerical schemes and algorithms as well as selected applications in the following sections.

¹ www.flexi-project.org

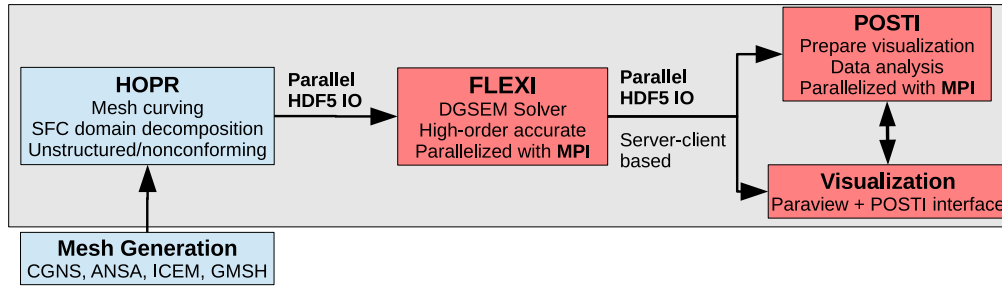


Fig. 1. Overview of the high order framework.

In this work, we focus on applications of *FLEXI* in the field of computational fluid dynamics (CFD). In particular, scale-resolving and time-accurate simulation methodologies such as large eddy simulation (LES) and direct numerical simulation (DNS) will be considered. A number of research groups apply DG methods to problems of such kind, see e.g. [19–22]. At its heart, *FLEXI* is a solver for general hyperbolic–parabolic conservation laws. For instance, the solver *Fluxo*² is a spin-off from *FLEXI* and is used for applications in astro- and plasmaphysics, solving the equations of magnetohydrodynamics including non-conservative terms. Its main focus is on the split-form DG formulation (see Section 3.3) and entropy stability [23].

2. Framework overview

The overall *FLEXI* framework consists of open-source tools for pre-processing and generation of high order meshes, the CFD solver itself and a post-processing and visualization suite, see Fig. 1. In the following sections, we will discuss each of the parts and give an overview over their methods and features.

2.1. HOPR: Mesh generation tool

In this section, we give an overview of the high order mesh preprocessor *HOPR*, which generates the mesh input file for *FLEXI*. Documentation and tutorials for *HOPR* can be found under www.hopr-project.org and details of the mesh curving techniques are given in [24,25].

When using high order methods, wall boundary conditions at curved geometries need a high order representation to maintain the high order accuracy of the method [18]. Especially CFD applications have complex geometries and pose high requirements for the quality of geometry representation. While the open-source software GMSH [26] is constantly improving the generation of high order meshes, it is still a topic of ongoing research [27–29]. In *HOPR*, the main approach is to rely on linear meshes from standard grid generators and give the user different techniques to curve the mesh. The curved mesh geometry is represented using polynomials of degree N_{geo} , which can be chosen depending on the considered case.

First, we present the high order mesh format that supports direct parallel read-in and meshes with curved non-conforming element interfaces. Further, we discuss the main curving techniques that are summarized in the flowchart in Fig. 2.

In *HOPR*, simple multi-block meshes can also be generated internally, without the need to rely on external grid generators. Such meshes can include the use of periodic boundary conditions and non-conforming element interfaces. In addition, the mesh can be curved a-posteriori via user-defined mappings. A detailed explanation of these two features will not be given here, but can be found in the tutorials on the webpage.

HOPR supports both tetrahedral and hexahedral mesh elements, such that different solvers can use the resulting mesh files. The meshes used by *FLEXI* require pure hexahedral elements, and geometrical flexibility is provided by the support for hanging nodes, see Section 3.2.

2.1.1. Mesh format

The mesh format in *HOPR* is designed for fast parallel read-in in *FLEXI*. Therefore, the mesh file is written in binary HDF5 format [30]. An important feature is that the elements are ordered along a space-filling curve (SFC), providing a direct domain decomposition during parallel read-in. That means one can directly start the parallel computation with an arbitrary number of domains ($\geq$ number of elements) and always read the same mesh file. The element list is simply divided by the number of domains, and each domain is associated with a contiguous range of elements. The SFC curve guarantees that these elements are, on average, in proximity of each other. An example is shown in Fig. 3 for an unstructured mesh, with two different domain decompositions based on the same *HOPR* mesh file. In *FLEXI*, the same element ordering is used to write the restart files.

² <https://github.com/project-fluxo/>

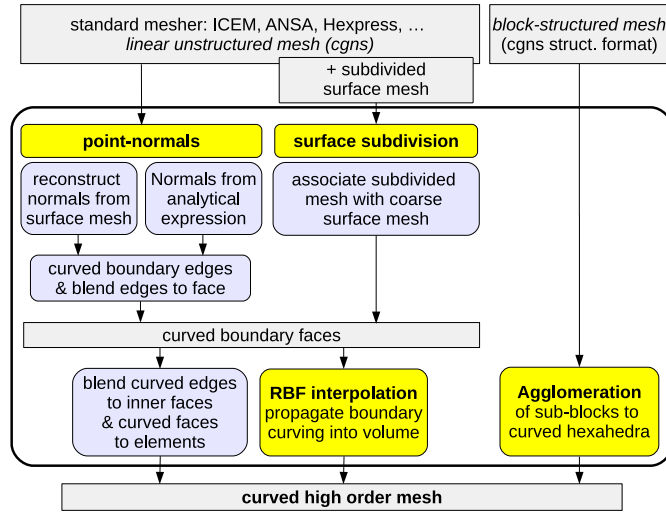


Fig. 2. Flowchart of the curved mesh generation process in HOPR.

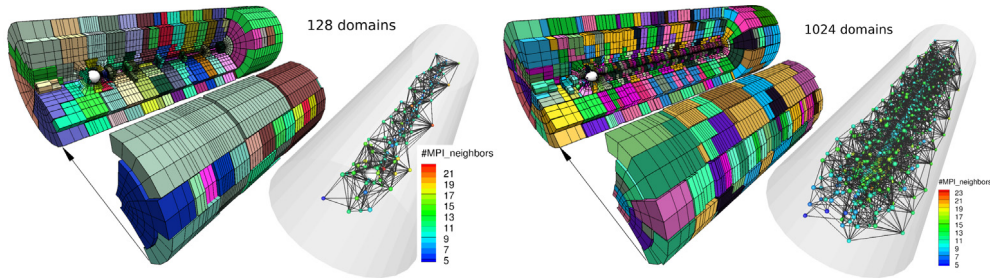


Fig. 3. Space-filling curve based domain decomposition of an unstructured mesh around a sphere, with communication graph between the domains, for 128 domains of 165–166 elements (left) and 1024 domains of 20–21 elements (right), from [24].

In contrast to standard mesh formats, which provide the element node connectivity list and the node list separately, the *HOPR* mesh format provides direct neighbor connectivity information of the element sides (conforming and non-conforming) and the element node information (index and (x, y, z) position) as a package per element. Hence, for a given range of elements, non-overlapping contiguous parts of the arrays can be read from file in parallel. As a consequence, the coordinates of the same physical nodes are stored in more than one location, but remain associated by a unique global node index.

2.1.2. Surface curving

In *HOPR*, there are two main strategies to curve the boundary surfaces, the ‘surface-subdivision’ and the ‘point-normal’ approach [25]. The surface-subdivision approach is the most convenient and accurate, but it needs an additional refined surface mesh. As shown in Fig. 4, a subdivided mesh was created with the mesh generator ANSA®, with the new points lying on the exact boundary surface, and the coarse mesh points being unchanged. Given the polynomial degree of the surface mapping ($N_{\text{geo}} = 2, 4, 8$) that corresponds to the number of refinement steps, *HOPR* finds automatically the connection between each boundary face and its refined surface elements by making use of the connectivity information of the subdivided surface mesh. The curved face is then defined by an interpolation polynomial through the new points.

The point-normal approach, proposed in [31], relies on the reconstruction of the curved surface using only normal vectors at the grid points of the boundary of the linear volume mesh. The surface elements are then G^1 continuous at the corner nodes. Clearly, the most difficult part is to provide the normal vector. In *HOPR*, analytical expressions for simple geometric shapes are included, and it is easy to add other user-defined functions. If no analytical expression is available, the normal vector is reconstructed from the linear surface mesh, with a grid-size dependent approximation error. We also need to consider sharp edges at the intersections of surfaces, where two point-normals exist. To find these edges, curved boundary patches can be associated with a unique index, and a sharp edge is assumed between patches of different indices.

The process to generate the curved faces from the normals is depicted in Fig. 5. The tangential vectors of the face edges are constructed first, by projecting the straight edge into the tangential plane. If two point-normals are given (at sharp

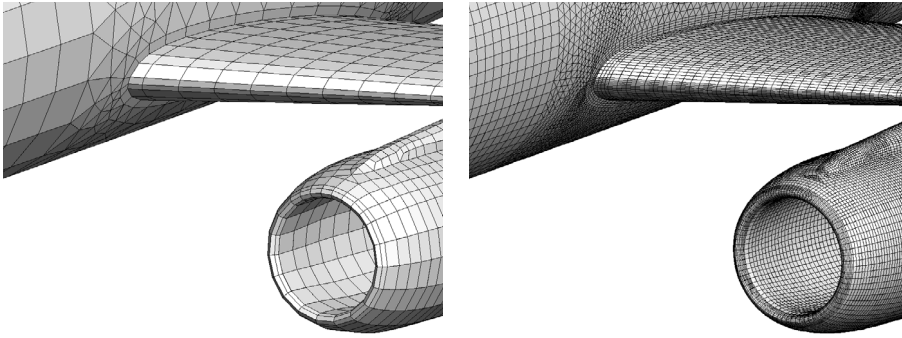


Fig. 4. Surface mesh of the DLR-F6 body-wing intersection, after two steps of isotropic refinement, from [25].

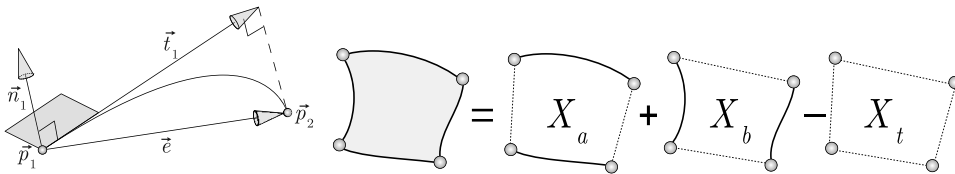


Fig. 5. Construction of curved edge tangential from point-normal vector (left) and blending of curved edges to curved surface (right), from [25].

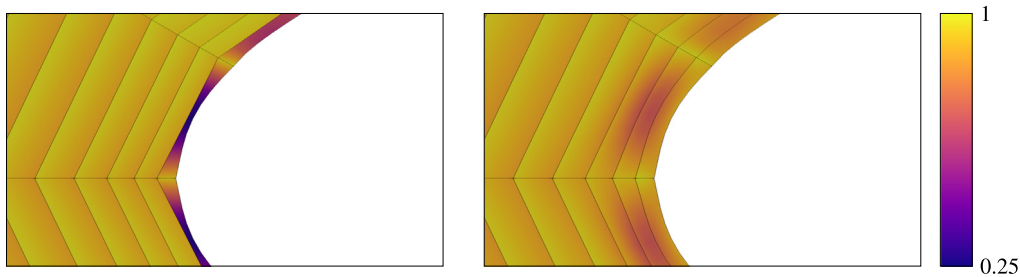


Fig. 6. Visualization of mesh and scaled Jacobian around the leading edge of a NACA0012 airfoil. Left: Curved surface without volume curving, right: curved surface and volume curving based on RBF interpolation.

edges), the direction of the tangential is found by a cross-product and its length again by projection onto the straight edge. The curved edge is a cubic polynomial, computed from the edge end points and the tangential vectors. In a following step, the curved faces are computed from a blending of the curved edges [32].

Once all boundary faces are curved, the 3D element mapping is computed using blending functions of curved edges and faces, for all elements that share at least one edge with a curved boundary face ('local curving'). If the elements are highly stretched, additional curving of inner elements would be required ('global curving') to avoid invalid cells, which is addressed in the next section.

2.1.3. Volume curving

The strategies presented in the last section allow to create curved boundary faces of the domain and curve only the elements adjacent to the boundary. No information about volume curving is present in those approaches. This can lead to serious problems, especially for highly anisotropic cells that are typical for boundary layer meshes. In such situations, applying the curving procedure only to a single side of a cell may lead to highly skewed or even inverted cells, which is demonstrated in the left plot in Fig. 6. The mesh is colored by the scaled Jacobian, defined as the ratio between the Jacobian of the high order mapping and its maximum value in each cell (see Section 3.1 for details on that quantity). Lower values indicate larger deformation, lowering the approximation quality in these regions. Since the physical boundaries are typically of high interest and their representation affects the solution quality globally, a way to remedy this loss is necessary. In *HOPR*, two possible solutions are available. The first is based on the interpolation of the surface curving to the volume based on radial basis function (RBF) interpolation, a method which can in general be used to interpolate scattered data, and is for instance used to deform meshes in response to a moving boundary [33]. Here, we interpolate the displacement of the curved surface into the volume, leading to improved mesh quality as can be seen on the right of Fig. 6. The main advantage of this approach is that it works with any type of mesh.

A different approach can be taken for specifically designed meshes. In general, the most simple way to generate a fully curved hexahedral mesh is by using block-structured meshes as input. The main requirement is that all blocks can be coarsened by a unique factor of n_b elements in all directions. Then, the high order element mapping is simply found via interpolation of an agglomerated block of $(n_b \times n_b \times n_b)$ cells [25]. Note that the polynomial degree of the mapping can be chosen independently from n_b , with $N_{\text{geo}} \leq n_b$.

2.2. FLEXI: DG solver

FLEXI is a general purpose solver for hyperbolic/parabolic systems following the method of lines, where the spatial operator is treated with a discontinuous Galerkin spectral element method (DGSEM), optionally combined with a subcell finite volume method to capture regions with shocks. High order explicit Runge–Kutta methods are used to integrate the semi-discrete DGSEM in time. Explicit time-integration in combination with the spectral element version of DG leads to a very efficient scheme that is well suited for massively parallel computing hardware. Details on the numerical schemes, methods and implementations will be given in Sections 3, 3.4 and 4. In this section, we will give an overview of its features.

2.2.1. Code design

In the context of this work, we present *FLEXI* with a focus on CFD, in particular for solving the unsteady, compressible Navier–Stokes equations. At its core however, *FLEXI* is intended to be a general purpose solver for hyperbolic–parabolic systems of partial differential equations. This goal requires a design with a focus on modularity, i.e. a collection of features or methods in functional units, which can be used and modified largely independently from each other. *FLEXI* and the framework are predominantly written in modern, procedural Fortran for its suitability and performance for scientific computing in HPC environments, as well as ease of implementation. In Fortran, code flexibility can be achieved by arranging functional units in modules, each with their own encapsulated set of variables. Within this concept, five major functional units exist in our solver:

1. The spatial operator, including the DG and FV discretization schemes
2. The temporal operator
3. Mesh preparation and handling
4. The equation system to be solved
5. Analysis and output routines

The inter-dependencies of these functional units are kept to a minimum. For example, the temporal integrator is essentially an ODE solver completely independent from all other units. This allows to e.g. substitute the currently used explicit time integration schemes with implicit ones without affecting other parts of the software [34,35]. The temporal discretization makes use of the spatial operators provided in a separate unit. Those spatial discretization operators are written for general hyperbolic and parabolic terms, and are further divided into modules that handle e.g. the surface and volume contributions of the operators, see Section 3.1 for details. A specific equation systems enters here by the definition of the flux functions, and an additional system to be solved can be added in a “plug and play” style, as again all code parts specific to the equation system (e.g. fluxes, time step restrictions, initial conditions, numerical flux functions) are organized in a single, interchangeable functional unit. In this functional unit, the user can specify the physical fluxes in Cartesian coordinates, easily implementing new equation systems. Features that are only available in certain systems (e.g. turbulence models for LES) can be implemented as sub-modules. In order to provide custom features specific to a user-defined case, a test case environment can be defined, which injects information at specific parts of the code. For example, within this test case, extra analysis routines or source terms can be defined, which are then invoked automatically.

The core features of *FLEXI*, including the hybrid scheme, are all implemented in the open-source version, meaning they are all designed to work together, and the interoperability is routinely tested using the regression check framework (see Section 5.1). The code base is of moderate size and a design goal is to keep the code structure simple, such that a small team or even a single person can understand and modify large parts of the code. This means that under certain circumstances performance optimizations are traded for simplicity. On the other hand, this has allowed us to modify *FLEXI* for a large range of specific applications, where the code base deviates largely from the version presented here. Some of those are mentioned in the next sections and the outlook.

2.2.2. Parallelization

FLEXI and its associated tools use a distributed memory parallelization based on the Message Passing Interface (MPI) standard. In this paradigm, each parallel process manages its own memory, and information interchange between processes occurs through messages being sent over the network only. Like for all element-based discretizations, a single grid cell is a natural conceptual, granular unit for parallelization, i.e. all domain decompositions are element-based. Due to the local approximation space in DG and the absence of continuity requirements on the solution across element interfaces, the communication pattern of DG schemes is side-based. Thus, for a computation in a domain of dimensionality d , only information of dimensionality $d - 1$ has to be exchanged. Furthermore, information only has to be exchanged between neighbors that share a face. In a communication sense, DG is thus comparable to FV schemes, with the important

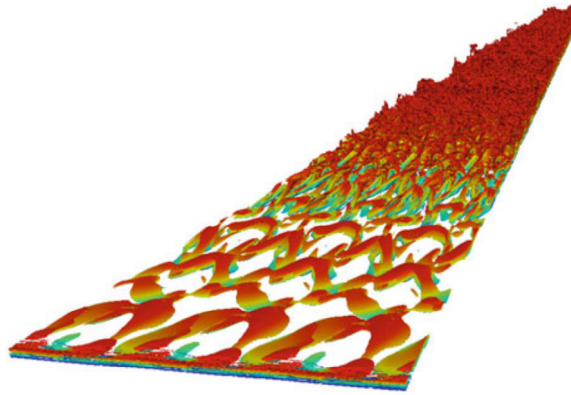


Fig. 7. Spatially-developing supersonic turbulent boundary layer up to $Re_\theta = 3878$ at $Ma = 2.67$, computed with 1.458 billion DOF per variable. λ_2 -Visualization of the turbulent structures along the flat plate colored by the streamwise velocity component u [36].

differences being the amount of data being sent (for high order DG) and the absence of volume work in FV, where the evaluation of the numerical flux requires the overwhelming amount of computational effort. In DG, where additional volume work exists, these local computations can be used for communication latency hiding. This high density of local operations and the small communication footprint make DG operators naturally parallel and allow for a rather simple parallelization of explicit-in-time DG schemes.

In *FLEXI*, we use the MPI concept of non-blocking communication extensively, which allows initiating a message transfer at the earliest possible instance but does not require immediate execution of this command. I/O of solution data is achieved by use of the *HDF5* library [30], where we additionally gather operations over a compute node to reduce file access for large scale computations. With the help of these features, simulations with over 1 billion degrees of freedom (DOF) on 100,000 cores and perfect strong scaling performance including I/O are possible. Fig. 7 shows a sample computation with *FLEXI*: a DNS of a supersonic turbulent boundary layer, reaching a momentum thickness based Reynolds number $Re_\theta = 3878$, computed with close to 1.5 billion DOFs per solution variable [36].

2.2.3. Code features

FLEXI contains a number of features, most of which are focused towards stability in underresolved scenarios and analytics for complex flows e.g. from direct aeroacoustic simulations. The most important ones can be grouped as follows:

1. Hybrid DG/FV operator: The spatial discretization in *FLEXI* is based on an element-wise formulation of the DG method of arbitrary order. As discussed before, high order schemes have favorable properties in smooth regions of the solution, but lack stability in underresolved regions, in particular when strong gradients occur, as is typical in compressible flows. These solution types thus require some form of shock capturing mechanism. In *FLEXI*, this is achieved by a reinterpretation of the DG polynomial in troubled cells as a finite volume solution on a structured subcell grid, i.e. a DG element is conceptually switched to a local, block-structured FV grid. On this grid, 1st or 2nd order FV total variation diminishing (TVD) schemes with slope limitation are then solved to provide element-local, accurate shock capturing. The inverse functionality is also included to revert back to the DG representation, once sufficient smoothness of the solution can be detected. Different, element-local indicators are available to judge the local smoothness and determine the operator choice. Due to the shared data structure of DG and FV and careful implementation, the hybrid method suffers almost no additional computational cost compared to pure DG. More details on this feature can be found in Section 3.4. Fig. 8 shows the application of this approach to 2D Riemann problems.
2. Non-linear stability: For under-resolved simulations of non-linear problems, stability becomes an important aspect of the numerical method. Specifically high order methods tend to show oscillatory behavior, ultimately leading to the crash of the simulation. Classical stabilization approaches are mostly based on the addition of artificial viscosity – either explicitly or implicitly, e.g. by using upwind operators. While the surface contribution in the DG method employs upwind Riemann solvers, the volume integral is completely free of numerical dissipation, such that the amount of added viscosity might not be enough to stabilize the simulation for even moderately under-resolved simulations. Adding viscosity to the volume might be a solution for the stability problem, but possibly also restricts accuracy. Thus, a method to stabilize the simulation without the need for excessive artificial viscosity is desirable. A possible way to achieve this is by using the so-called split form of the advection operator, in which aliasing errors can be avoided to guarantee preservation of kinetic energy (or entropy) at a discrete level [38]. Within *FLEXI*, several split variants are available. As an alternative stabilization method, consistent integration of the non-linear terms can also be selected [39].

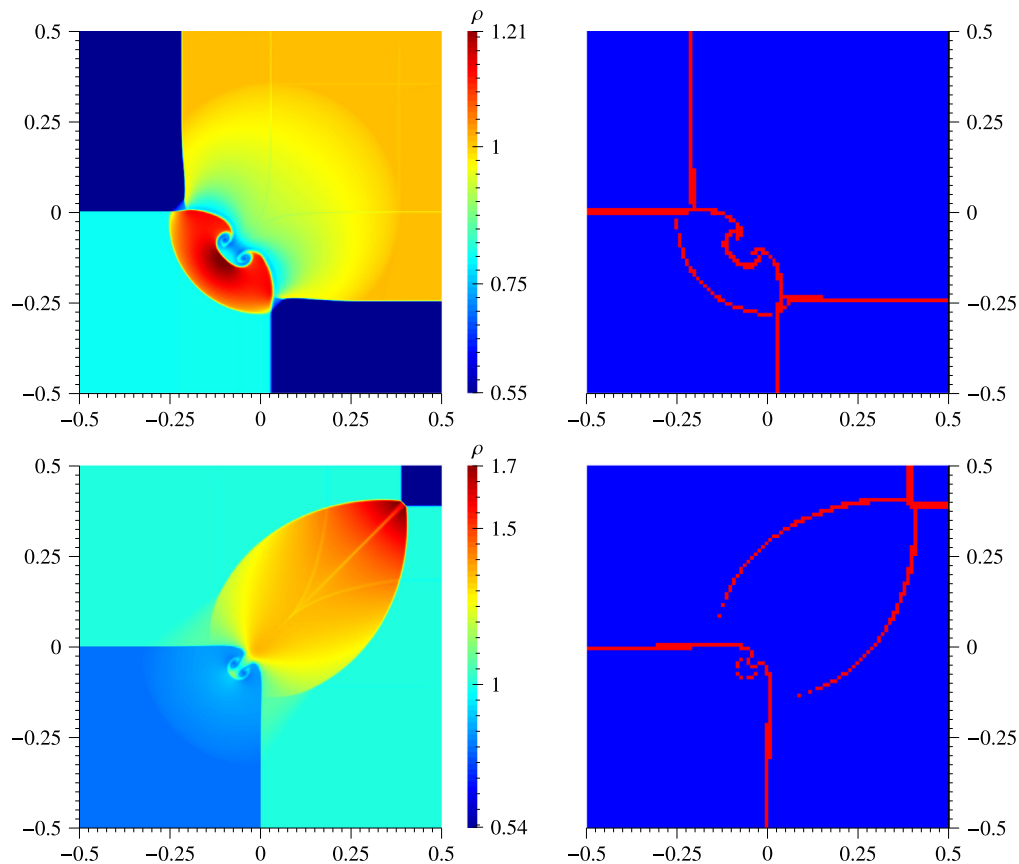


Fig. 8. Two dimensional Riemann problem, configurations E and F from [37]. Left: Density at final time. Right: FV (red) and DG (blue) elements. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3. **LES formulations:** As common in CFD, LES in *FLEXI* is based on the implicitly filtered form of the governing equations, i.e. the scale separation filter is determined by the cut-off behavior of the discretization. For well-resolved LES, implicit closure through the discretization operator, in particular the choice of the numerical flux function, is available [39–41]. A number of flux formulations for both the advective fluxes (e.g. Roe's approximate Riemann solver, Rusanov flux, HLL-type formulations [42]) as well as the viscous fluxes (i.e. two formulations according to Bassi and Rebay) [9,43] can be used. For typical LES resolutions that require additional modeling, the Smagorinsky, the σ and Vreman's eddy viscosity models can be selected [44–46]. Common filtering operations like volume or surface filtering with sharp or smooth filter kernels are also available and can thus be used for filter-based LES or advanced multiscale models like Variational Multiscale (VMS) approaches or other structural models [47]. In order to reduce the resolution requirements for wall-bounded flows at high Reynolds numbers, *FLEXI* uses a wall-stress model according to Larsson [48]. Initial solutions for the LES can be obtained by a Reynolds-averaged Navier–Stokes (RANS) formulation with a Spalart–Allmaras model [49] using pseudo-time stepping.
4. **Boundary Conditions:** Boundary conditions in DG methods are typically enforced weakly through the choice of the corresponding interface flux. This choice leads to a more stable and accurate solution at the boundaries [50], and makes implementation of a new boundary type rather easy. Standard boundary conditions for compressible flows are available in *FLEXI*, among them Dirichlet conditions, super- and subsonic inlet and outlet conditions, solutions to a Blasius boundary layer profile and the possibility to read in a given state from file, e.g. when using information from a precursor RANS computation as far field boundaries. Generating synthetic inflow turbulence for boundary layer computations is achieved by the anisotropic linear forcing (ALF) formulation [51]. In order to avoid reflection at the outflow faces in particular for aeroacoustic simulations, an absorbing layer variant called a sponge layer is available. In this method, a retarding volume forcing term is introduced in the vicinity of the outflow, which dampens the solution towards a baseflow. A flexible and general method to determine a suitable base flow is to generate it from a moving time-average of the solution which can be computed by an exponential temporal filter [52].
5. **Flow analytics:** *FLEXI* features a number of flow analysis options. The most basic one is that the solution (or any derived quantity) can be written to a solution file at a specified interval. Also, error norms with respect to a given

exact function can be computed and stored at any time. Other information gathered and reported are the time history of the forces acting on immersed bodies, the velocity slip at solid walls, the bulk quantities and overall balances. For more advanced analysis of turbulent flows, *FLEXI* employs an on-the-fly time-averaging procedure for arbitrary linear and non-linear quantities. This information is stored at the same instance as the solution files, and can be reassembled in post-processing to produce volume information about the fluctuating quantities, e.g. the Reynolds stresses. For time-dependent solutions, *FLEXI* also supports data probes or record points, where the time accurate history of the full solution vector is stored for later analysis.

2.3. POSTI: Post-processing suite

Since the main focus of *FLEXI* is on unsteady scale-resolving computations, the output of a simulation potentially consists of a large amount of files each containing the solution variables on tens or hundreds of million degrees of freedom for a single time step. Besides demanding large resources for storage, post-processing such amounts of data becomes a real challenge. The classical approach to visualization includes a conversion step, where the data from the file format native to the solver is transformed to an alternative format suitable for the visualization tool of choice. This not only introduces the need for additional storage space for the converted files, but also neither the conversion process nor the visualization tool itself might be suitable for parallel execution, which could lead to a prohibitively large cost of visualizing simulation results. Another important aspect to consider is that the large majority of available tools cannot efficiently or accurately handle the high order data provided by *FLEXI*.

Thus, the visualization process within the *FLEXI* framework follows a custom approach. We use the open-source software ParaView [53] as our primary visualization tool. One of the main features of ParaView is the capability to use remote computing resources for visualization in a server-client environment. Thus, a user can connect from a local workstation to e.g. a server on the cluster system used to run a large-scale simulation and directly use the processing power available there. Instead of converting the native HDF5 file format of *FLEXI* to a format suitable for ParaView, a custom read-in plugin has been created, which allows to directly open the simulation results without any prior conversion. This reader effectively provides an interface to a fully parallel visualization software, specifically designed for our framework. Since it is incorporated in the software package, it can make use of routines originally designed for the solver itself, e.g. domain decomposition algorithms or the calculation of gradients. It is also fully aware of the high order nature of the solution, which is used to super sample the solution and visualize that version on ParaView. Additionally, the equation system used for the calculation is known to the software, and it provides routines to calculate derived quantities from the solution variables, e.g. temperature or vorticity for the Navier–Stokes equations. The results will then be returned directly to ParaView without first writing them to the hard drive.

As an alternative, the visualization tool itself can be used to follow the classical approach and convert the *FLEXI* file format to the native ParaView file format. Next to the visualization software, the post-processing suite *POSTI* consists of several other programs. Those include tools to perform temporal or spatial averaging, analysis based on Fourier transformation, direct mode decomposition and several other, smaller programs. As a side project, we created a pipeline [54] which allows to render results from *FLEXI* in the open-source 3D computer graphics toolset Blender [55]. It can be used to present 3D simulation results in a pleasing visual style, and some examples can be found in this article.

3. Numerical methods

3.1. Discontinuous Galerkin spectral element method

In the following section, the DGSEM as it is implemented in *FLEXI* will be derived for a general system of hyperbolic–parabolic conservation laws for the conserved variables $\vec{U}$ defined on $\Omega \subseteq \mathbb{R}^3$:

$$\frac{\partial \vec{U}(\vec{x}, t)}{\partial t} + \nabla \cdot \left(\vec{F}^c(\vec{U}) - \vec{F}^v(\vec{U}, \nabla \vec{U}) \right) = 0, \quad (1)$$

with convective and viscous fluxes $\vec{F}^c$ and $\vec{F}^v$ respectively. We combine them into a single flux $\vec{F}$ to keep the notation compact.

The domain Ω is subdivided into $K \in \mathbb{N}$ non-overlapping and (for the moment) conforming hexahedral elements e_K , each bounded by six curved faces $\vec{\Gamma}_i$, $i = 1, 2, 3, 4, 5, 6$, as sketched in Fig. 9. The geometry of the faces $\vec{\Gamma}_i(r, s)$, $(r, s) \in [-1, 1]^2$ is approximated as tensor products of one-dimensional Lagrange interpolating polynomials up to degree N_{geo} :

$$\vec{\Gamma}_i(r, s) := \sum_{j,k=0}^{N_{\text{geo}}} \vec{\Gamma}_i(r_j, s_k) \ell_j(r) \ell_k(s). \quad (2)$$

The usual definition of the Lagrange polynomials is

$$\ell_j(r) = \prod_{i=0, i \neq j}^{N_{\text{geo}}} \frac{r - r_i}{r_j - r_i}. \quad (3)$$

The nodal points r_i are taken from either Legendre–Gauss (LG) or Legendre–Gauss–Lobatto (LGL) quadrature.

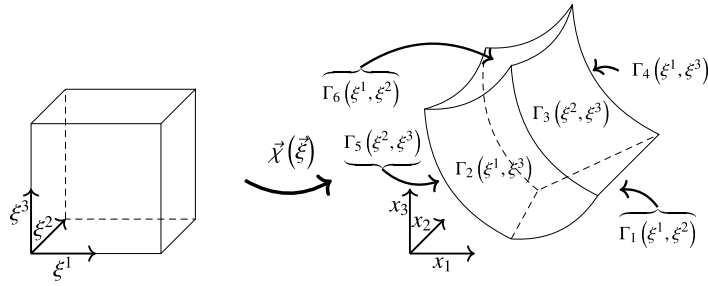


Fig. 9. Left the reference element $E = [-1, 1]^3$ and on the right a general hexahedral element e_k with the curved faces $\tilde{\Gamma}_i$. The mapping $\tilde{\chi}(\tilde{\xi})$ connects E and e_k .

The DGSEM will be constructed on a reference cube, such that all operators only need to be defined once, regardless of the actual shape of the considered element. Thus, a transfinite map $\tilde{x} = \tilde{\chi}(\tilde{\xi}, t)$ from the reference cube $E = [-1, 1]^3$ in *computational* space $\tilde{\xi} = [\xi^1, \xi^2, \xi^3]^T$ onto the element in *physical* space $\tilde{x} = [x_1, x_2, x_3]^T$ is introduced. We then define the Jacobian of the mapping $\mathcal{J}$ as the determinant of the Jacobian matrix $\nabla_{\tilde{\xi}} \tilde{\chi}$. The operator $\nabla_{\tilde{\xi}}$ represents the nabla operator evaluated in the computational coordinates. For the required transformation between physical and reference space, the contravariant basis vectors $\mathcal{J} \tilde{a}^i$ (also called metric terms) are introduced, calculated in a guaranteed discretely divergence free (so called curl) form [56]:

$$\mathcal{J} \tilde{a}_n^i = -\hat{x}_i \cdot \nabla_{\tilde{\xi}} \times (x_i \nabla_{\tilde{\xi}} x_m), \quad i = 1, 2, 3, \quad n = 1, 2, 3, \quad (n, m, l) \text{ cyclic}, \quad (4)$$

where $\hat{x}_i$ is the unit vector in the i th Cartesian direction. This formulation ensures that the so-called metric identities are fulfilled, which is necessary to achieve free-stream preservation.

As is detailed in [25], the mapping is constructed by blending the curved boundary faces together. This ensures that the map $\tilde{\chi}$ itself is a polynomial of degree N_{geo} or less, and that required derivatives can be computed by derivation of that polynomial.

The conservation law (1) can now be formulated in the reference cube. Inserting the transformation rule for the divergence

$$\nabla \cdot \tilde{F} = \frac{1}{\mathcal{J}} \sum_{i=1}^3 \frac{\partial \mathcal{J} \tilde{a}^i \cdot \tilde{F}}{\partial \xi^i}, \quad (5)$$

yields

$$\mathcal{J} \frac{\partial \tilde{U}(\tilde{x}, t)}{\partial t} + \nabla_{\tilde{\xi}} \cdot \tilde{F} = 0, \quad (6)$$

with the so-called contravariant fluxes $\tilde{F}$, which are defined as

$$\tilde{F}^i = \mathcal{J} \tilde{a}^i \cdot \tilde{F}, \quad i = 1, \dots, 3. \quad (7)$$

We can obtain the weak formulation of the conservation law in reference coordinates by multiplying (6) with an arbitrary test function $\phi = \phi(\tilde{\xi})$, integration over the reference element and an application of Green's identity to separate interior and boundary contributions to obtain

$$\left\langle \mathcal{J} \frac{\partial \tilde{U}}{\partial t}, \phi \right\rangle + \iint_{\partial E} \phi (\tilde{F} \cdot \mathcal{N})^* dS - \langle \tilde{F}, \nabla_{\tilde{\xi}} \phi \rangle = 0, \quad (8)$$

where $\mathcal{N}$ is the outward pointing normal vector and $\langle \cdot, \cdot \rangle$ defines the inner product between two functions. Since no continuity is required between elements, numerical flux functions are used to approximate a unique normal flux $(\tilde{F} \cdot \mathcal{N})^*$ from the double-valued solution on the boundary.

Both the solution $\tilde{U}$ and the components of the contravariant fluxes $\tilde{F}^l$, $l = 1, 2, 3$ are approximated by tensor products of Lagrange polynomials of degree N on the reference element, e.g.

$$\tilde{U}(\xi^1, \xi^2, \xi^3, t) \approx \tilde{\mathbf{U}}(\xi^1, \xi^2, \xi^3, t) = \sum_{m,n,o=0}^N \tilde{\mathbf{U}}_{mno}(t) \ell_m(\xi^1) \ell_n(\xi^2) \ell_o(\xi^3), \quad (9)$$

where the bold font indicates the polynomial interpolation. The notation $\tilde{\mathbf{U}}_{mno}(t)$ stands for the degree of freedom associated with the basis function $\psi_{mno} = \ell_m(\xi^1) \ell_n(\xi^2) \ell_o(\xi^3)$. For the chosen nodal basis, the degrees of freedom are also identical to the value of the interpolation at the position $(\xi_m^1, \xi_n^2, \xi_o^3)$. The nodes are again chosen from either LG or LGL

quadrature. To determine the nodal values of the contravariant flux components $\tilde{\mathcal{F}}_{mno}^j$, the physical flux is evaluated using the interpolated solution and multiplied by the weighted contravariant basis vectors, transforming it to computational space:

$$\tilde{\mathcal{F}}_{mno}^j(t) = \sum_{d=1}^3 \mathcal{J} a_d^j(\xi^1, \xi^2, \xi^3, t) \tilde{F}^d(\vec{\mathbf{u}}(\xi^1, \xi^2, \xi^3, t)). \quad (10)$$

The polynomial approximations can now be inserted into the weak form (8). Integrals will be evaluated using Gauss–Legendre quadrature with collocation of the interpolation and quadrature nodes. The basis functions are also used as test functions, following the Galerkin ansatz.

Time derivative integral

Substituting the polynomial approximation into the weak form (8) and replacing the integrals by Gauss–Legendre quadrature yield for the first term, containing the time derivative:

$$\left\langle \mathcal{J} \frac{\partial \vec{\mathbf{u}}}{\partial t}, \phi \right\rangle = \frac{\partial}{\partial t} \sum_{\alpha, \beta, \gamma} \mathcal{J}(\xi_{\alpha\beta\gamma}) \sum_{m, n, o} \left(\vec{\mathbf{u}}_{mno}(t) \ell_m(\xi_{\alpha}^1) \ell_n(\xi_{\beta}^2) \ell_o(\xi_{\gamma}^3) \right) \psi_{ijk}(\xi_{\alpha\beta\gamma}) \omega_{\alpha} \omega_{\beta} \omega_{\gamma} \quad \forall i, j, k. \quad (11)$$

Here, $\{\omega_i\}_{i=0}^N$ are the weights associated with the quadrature rule and $\{\xi_i\}_{i=0}^N$ the position of the nodes. Since the Lagrange polynomials satisfy the cardinal property, the above equation simplifies to

$$\left\langle \mathcal{J} \frac{\partial \vec{\mathbf{u}}}{\partial t}, \phi \right\rangle = \mathcal{J}(\xi_{ijk}) \frac{\partial \vec{\mathbf{u}}_{ijk}}{\partial t} \omega_i \omega_j \omega_k \quad \forall i, j, k. \quad (12)$$

Our choice of basis functions and the collocation approach lead to significant simplifications of the scheme. We obtain a diagonal mass matrix $M_{ij} = \langle \ell_i, \ell_j \rangle$, a manifestation of the discrete orthogonality of the basis functions. It should be noted that (for a constant Jacobian) the mass matrix is exactly computed for LG nodes, since the quadrature in this case is exact up to a degree $2N + 1$, and the product of the two functions is of degree $2N$. Quadrature on LGL nodes is only exact up to degree $2N - 1$, and the analytical mass matrix for LGL nodes would thus not be diagonal. The discrete version is, which is referred to as *mass lumping*, a property that has an influence on dispersion and dissipation characteristics and which was investigated in [15].

Volume integral

The rightmost term in the weak formulation (8) is the scalar product of the contravariant fluxes with the gradient of the test function, compactly written as

$$\langle \vec{\mathcal{F}}, \nabla_{\xi} \phi \rangle = \sum_{d=1}^3 \left\langle \vec{\mathcal{F}}^d, \frac{\partial \phi}{\partial \xi^d} \right\rangle. \quad (13)$$

Only the first entry in the sum ($d = 1$) will be considered in the following, since they are all of the same structure. Applying Gauss quadrature, substituting the polynomial approximation and using the Lagrange property leads to

$$\left\langle \vec{\mathcal{F}}^1, \frac{\partial \phi}{\partial \xi^1} \right\rangle = \omega_j \omega_k \sum_{m=0}^N \tilde{\mathcal{F}}_{mjk}^1 \frac{\partial \ell_i(\xi^1)}{\partial \xi^1} \Big|_{\xi=\xi_m^1} \omega_m \quad \forall i, j, k. \quad (14)$$

We can define the differentiation matrix

$$D_{rs} = \frac{\partial \ell_s(\xi)}{\partial \xi} \Big|_{\xi=\xi_r}, \quad r, s = 0, \dots, N, \quad (15)$$

containing the derivatives of the Lagrange polynomials at the interpolation nodes. This allows us to write the sum over the derivative of the Lagrange polynomials as a matrix vector product, which can be efficiently implemented:

$$\sum_{m=0}^N \frac{\partial \ell_i(\xi^1)}{\partial \xi^1} \Big|_{\xi=\xi_m^1} \omega_m = \sum_{m=0}^N D_{mi} \omega_m. \quad (16)$$

Using this notation, the volume integral can be written as

$$\langle \vec{\mathcal{F}}, \nabla_{\xi} \phi \rangle = \omega_j \omega_k \sum_{m=0}^N \tilde{\mathcal{F}}_{mjk}^1 D_{mi} \omega_m + \omega_i \omega_k \sum_{n=0}^N \tilde{\mathcal{F}}_{ink}^2 D_{nj} \omega_n + \omega_i \omega_j \sum_{o=0}^N \tilde{\mathcal{F}}_{ijo}^3 D_{ok} \omega_o \quad \forall i, j, k. \quad (17)$$

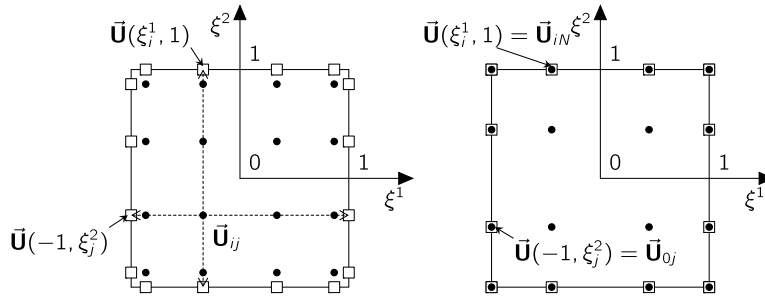


Fig. 10. Two-dimensional sketch of the location of the interpolation nodes in the volume $\bullet$ and on the surface $\square$, both for LG nodes (left) and LGL nodes (right) with $N = 3$.

Surface integral

The second term of the weak form (8) consists of the surface integral, which can be evaluated on all six sides of the reference cube independently and summed up subsequently:

$$\begin{aligned} \iint_{\partial E} \phi(\vec{F} \cdot \mathcal{N})^* dS = & \int_{-1}^1 \int_{-1}^1 \phi(\vec{F} \cdot \mathcal{N})^* d\xi_2 d\xi_3 \Big|_{\xi_1=1} + \int_{-1}^1 \int_{-1}^1 \phi(\vec{F} \cdot \mathcal{N})^* d\xi_2 d\xi_3 \Big|_{\xi_1=-1} \\ & + \int_{-1}^1 \int_{-1}^1 \phi(\vec{F} \cdot \mathcal{N})^* d\xi_1 d\xi_3 \Big|_{\xi_2=1} + \int_{-1}^1 \int_{-1}^1 \phi(\vec{F} \cdot \mathcal{N})^* d\xi_1 d\xi_3 \Big|_{\xi_2=-1} \\ & + \int_{-1}^1 \int_{-1}^1 \phi(\vec{F} \cdot \mathcal{N})^* d\xi_1 d\xi_2 \Big|_{\xi_3=1} + \int_{-1}^1 \int_{-1}^1 \phi(\vec{F} \cdot \mathcal{N})^* d\xi_1 d\xi_2 \Big|_{\xi_3=-1}. \end{aligned} \quad (18)$$

The Riemann solvers used for the numerical fluxes are in general implemented to compute the flux in the normal direction in physical space on the surface $(\vec{F} \cdot \hat{n})^*$, so we need to relate those quantities to the computational system. This is done by introducing the surface element (the norm of the non-normalized physical unit vector)

$$\hat{s} = \sqrt{\sum_{d=1}^3 (\mathcal{J}a_d^1(1, \xi_2, \xi_3))^2}, \quad (19)$$

which is used to write

$$(\vec{F} \cdot \mathcal{N})^* = (\vec{F} \cdot \hat{n})^* \hat{s} = \vec{f}^*(\vec{U}_L, \vec{U}_R, \hat{n}) \hat{s}. \quad (20)$$

By $\vec{f}^*$ we denote the Riemann solver that takes the double-valued solution at the left and the right side ($\vec{U}_L$ and $\vec{U}_R$, respectively) at an interface (along with the physical normal unit vector) as arguments and returns a unique flux in the direction of that normal vector. The left state is obtained from the polynomial interpolation of the solution in the mesh element currently considered, while the right state is obtained from the solution in the adjacent element in the considered direction. Riemann solvers only handle the convective part of the fluxes, the treatment of the viscous part is described later.

The numerical flux is again approximated using Lagrange interpolation, e.g. on the side $\xi^1 = 1$

$$(\vec{F} \cdot \mathcal{N})^* = \sum_{m,n=0}^N \left[\vec{f}^*(\vec{U}_L, \vec{U}_R, \vec{v}, \hat{n}) \hat{s} \right]_{m,n}^{+\xi^1} \ell_m(\xi^2) \ell_n(\xi^3), \quad (21)$$

Here, the superscript $+\xi^1$ indicates the evaluation at the face where $\xi^1 = 1$ and the subscript m, n identifies the m, n th Gauss node on the cell boundary. For LGL nodes, since the boundaries are included in the set of nodes, the interface nodal points coincide with nodes from the volume, and the solution can be directly obtained from the degrees of freedom of the Lagrange interpolation. For LG nodes, the solution at the face must be obtained by evaluating the interpolation at $\xi^1 = 1$. Exploiting the tensor product nature of the interpolation, this reduces to a one-dimensional operation:

$$\vec{U}_{m,n}^{+\xi^1} = \sum_{o=0}^N \vec{U}_{mno} \ell_o(1) \quad (22)$$

A sketch of the situation for both LG and LGL nodes is given in Fig. 10, in two dimensions.

We now insert the numerical flux (21) into the corresponding part of the surface integral (18), again apply Gauss quadrature, use the Galerkin ansatz and make use of the Lagrange property:

$$\int_{-1}^1 \int_{-1}^1 \phi (\vec{\mathcal{F}} \cdot \mathcal{N})^* d\xi_2 d\xi_3 \big|_{\xi_1=1} = [\vec{f}^* \hat{s}]_{j,k}^{+\xi^1} \ell_i(1) \omega_j \omega_k \quad \forall i, j, k. \quad (23)$$

We omit the arguments of the Riemann solver to shorten the notation. Repeating the same procedure for the remaining five sides yields the complete surface integral:

$$\begin{aligned} \iint_{\partial E} \phi (\vec{\mathcal{F}} \cdot \mathcal{N})^* dS &= \left([\vec{f}^* \hat{s}]_{j,k}^{+\xi^1} \ell_i(1) + [\vec{f}^* \hat{s}]_{j,k}^{-\xi^1} \ell_i(-1) \right) \omega_j \omega_k \\ &+ \left([\vec{f}^* \hat{s}]_{i,k}^{+\xi^2} \ell_j(1) + [\vec{f}^* \hat{s}]_{i,k}^{-\xi^2} \ell_j(-1) \right) \omega_i \omega_k \\ &+ \left([\vec{f}^* \hat{s}]_{i,j}^{+\xi^3} \ell_k(1) + [\vec{f}^* \hat{s}]_{i,j}^{-\xi^3} \ell_k(-1) \right) \omega_i \omega_j \quad \forall i, j, k. \end{aligned} \quad (24)$$

Semi-discrete formulation

Now each part of the weak formulation (8) is discretized, and we can combine the results and gain the semi-discrete version of the DGSEM, with a still continuous time:

$$\begin{aligned} \frac{\partial \vec{U}_{ijk}}{\partial t} &= -\frac{1}{\mathcal{J}_{ijk}} \left[\sum_{\alpha=0}^N \vec{\mathcal{F}}_{\alpha jk}^1 \hat{D}_{i\alpha} + \left([\vec{f}^* \hat{s}]_{j,k}^{+\xi^1} \hat{\ell}_i(1) + [\vec{f}^* \hat{s}]_{j,k}^{-\xi^1} \hat{\ell}_i(-1) \right) \right. \\ &+ \sum_{\beta=0}^N \vec{\mathcal{F}}_{i\beta k}^2 \hat{D}_{j\beta} + \left([\vec{f}^* \hat{s}]_{i,k}^{+\xi^2} \hat{\ell}_j(1) + [\vec{f}^* \hat{s}]_{i,k}^{-\xi^2} \hat{\ell}_j(-1) \right) \\ &\left. + \sum_{\gamma=0}^N \vec{\mathcal{F}}_{ij\gamma}^3 \hat{D}_{k\gamma} + \left([\vec{f}^* \hat{s}]_{i,j}^{+\xi^3} \hat{\ell}_k(1) + [\vec{f}^* \hat{s}]_{i,j}^{-\xi^3} \hat{\ell}_k(-1) \right) \right] \quad \forall i, j, k. \end{aligned} \quad (25)$$

The operator can be split into a *volume integral* (left half) and a *surface integral* (right half). For the surface integral, the *numerical flux function* must first be computed. Those three procedures are the main working points of the DG method. Choosing the tensor product basis directly leads to a tensor product structure of the operator itself: Each of the three lines correspond to the application of a one-dimensional DG operator in one of the reference directions. Thus, we can make use of the precomputed one-dimensional operators

$$\hat{\ell}_i = \frac{\ell_i}{\omega_i} \quad \text{and} \quad \hat{D}_{ij} = -\frac{\omega_i}{\omega_j} D_{ji}, \quad i, j = 0, \dots, N, \quad (26)$$

for an efficient computation.

Strong formulation

If Green's identity is applied once more to the weak form (8), and only using the interior contribution $\vec{\mathcal{F}}^{int}$ in the newly emerging surface integral, this leads to

$$\left\langle \mathcal{J} \frac{\partial \vec{U}}{\partial t}, \phi \right\rangle + \iint_{\partial E} \phi ((\vec{\mathcal{F}} \cdot \mathcal{N})^* - \vec{\mathcal{F}}^{int} \cdot \mathcal{N}) dS + \langle \nabla_{\xi} \vec{\mathcal{F}}, \phi \rangle = 0. \quad (27)$$

In this form, the volume flux still needs to be differentiable, and it is thus referred to as *strong form*. It consists of the local residual of the equation (the last term) and a surface penalty term, that vanishes when the solution is continuous at the boundary.

Repeating the discretizations steps for this version of the equations leads to the semi-discrete formulation

$$\begin{aligned} \frac{\partial \vec{U}_{ijk}}{\partial t} &= \\ &-\frac{1}{\mathcal{J}_{ijk}} \left[\sum_{\alpha=0}^N \vec{\mathcal{F}}_{\alpha jk}^1 D_{i\alpha} + \left([\vec{f}^* \hat{s} - \vec{\mathcal{F}}^1]_{j,k}^{+\xi^1} \hat{\ell}_i(1) + [\vec{f}^* \hat{s} - \vec{\mathcal{F}}^1]_{j,k}^{-\xi^1} \hat{\ell}_i(-1) \right) \right. \\ &+ \sum_{\beta=0}^N \vec{\mathcal{F}}_{i\beta k}^2 D_{j\beta} + \left([\vec{f}^* \hat{s} - \vec{\mathcal{F}}^2]_{i,k}^{+\xi^2} \hat{\ell}_j(1) + [\vec{f}^* \hat{s} - \vec{\mathcal{F}}^2]_{i,k}^{-\xi^2} \hat{\ell}_j(-1) \right) \\ &\left. + \sum_{\gamma=0}^N \vec{\mathcal{F}}_{ij\gamma}^3 D_{k\gamma} + \left([\vec{f}^* \hat{s} - \vec{\mathcal{F}}^3]_{i,j}^{+\xi^3} \hat{\ell}_k(1) + [\vec{f}^* \hat{s} - \vec{\mathcal{F}}^3]_{i,j}^{-\xi^3} \hat{\ell}_k(-1) \right) \right] \end{aligned}$$

$$+ \sum_{\gamma=0}^N \bar{\mathcal{F}}_{ij\gamma}^3 D_{k\gamma} + \left(\left[\bar{f}^* \hat{s} - \bar{\mathcal{F}}^3 \right]_{i,j}^{+\xi^3} \hat{\ell}_k(1) + \left[\bar{f}^* \hat{s} - \bar{\mathcal{F}}^3 \right]_{i,j}^{-\xi^3} \hat{\ell}_k(-1) \right) \Big] \\ \forall i, j, k. \quad (28)$$

For the volume integral, the difference between weak and strong form is simply that the strong form uses directly the D matrix instead of $\hat{D}$, meaning the same algorithm can be used for both of them. The interior surface contribution must be computed by interpolating the volume flux to the boundary, introducing additional operations. Analytically and also discretely, the weak and the strong formulation are identical for DGSEM for LGL quadrature, and for LG quadrature with special care for the internal fluxes [57]. The main use for the strong form is that it can be used as a starting point for the construction of non-linear stable schemes, as will be described in Section 3.3.

Approximation of second order derivatives

To handle viscous fluxes, that also depend on the gradients of the solution, we follow a method by Bassi and Rebay [9], now known as BR1. While FLEXI also supports the BR2 method [43], we restrict ourselves here to the details of BR1. For this procedure, known as *lifting*, we introduce an additional gradient quantity

$$\vec{g}^d = \frac{\partial \vec{U}}{\partial x_d}, \quad d = 1, 2, 3. \quad (29)$$

Those are used to express second-order equations as a system of first-order equations:

$$\frac{\partial \vec{U}(\vec{x}, t)}{\partial t} + \nabla \cdot \left(\vec{F}^c(\vec{U}) - \vec{F}^v(\vec{U}, \vec{g}) \right) = 0, \quad (30)$$

$$\vec{g}^d - \frac{\partial \vec{U}}{\partial x_d} = 0, \quad d = 1, 2, 3. \quad (31)$$

An additional $3m$ (in the case of three spatial dimensions) gradient equations must now be solved for m variables that need to be lifted. We note that depending on the viscous fluxes, not all of the conservative variables might be lifted, or alternative sets of variables can be used.

We apply the same discretization steps to the gradient equations, starting with the transformation to the computational space:

$$\vec{g}^d - \frac{1}{J} \nabla_{\xi} \cdot \vec{U}^d = 0, \quad (32)$$

with the contravariant solution in direction d

$$\vec{U}^d = (\mathcal{J}a_d^1, \mathcal{J}a_d^2, \mathcal{J}a_d^3)^T \vec{U}. \quad (33)$$

The weak formulation of Eq. (32) is again obtained by projection onto the space of test functions and applying Green's first identity

$$\langle \mathcal{J} \vec{g}^d, \phi \rangle - \iint_{\partial E} \phi (\vec{U}^d \cdot \mathcal{N})^* dS + \langle \vec{U}^d, \nabla_{\xi} \phi \rangle = 0. \quad (34)$$

The discretized version of the weak gradient equations then reads

$$\vec{g}_{ijk}^d = \frac{1}{J_{ijk}} \left[\sum_{\alpha=0}^N \vec{u}_{\alpha jk}^{1,d} \hat{D}_{i\alpha} + \left(\left[\vec{U}^* \hat{n}_d \hat{s} \right]_{j,k}^{+\xi^1} \hat{\ell}_i(1) + \left[\vec{U}^* \hat{n}_d \hat{s} \right]_{j,k}^{-\xi^1} \hat{\ell}_i(-1) \right) \right. \\ + \sum_{\beta=0}^N \vec{u}_{i\beta k}^{2,d} \hat{D}_{j\beta} + \left(\left[\vec{U}^* \hat{n}_d \hat{s} \right]_{i,k}^{+\xi^2} \hat{\ell}_j(1) + \left[\vec{U}^* \hat{n}_d \hat{s} \right]_{i,k}^{-\xi^2} \hat{\ell}_j(-1) \right) \\ \left. + \sum_{\gamma=0}^N \vec{u}_{ij\gamma}^{3,d} \hat{D}_{k\gamma} + \left(\left[\vec{U}^* \hat{n}_d \hat{s} \right]_{i,j}^{+\xi^3} \hat{\ell}_k(1) + \left[\vec{U}^* \hat{n}_d \hat{s} \right]_{i,j}^{-\xi^3} \hat{\ell}_k(-1) \right) \right] \quad \forall i, j, k. \quad (35)$$

The specific lifting procedure now must define the unique solution $\vec{U}^*$ used as the numerical surface flux, and how to calculate the numerical viscous flux at the cell boundaries for the main equations. The BR1 scheme simply uses the arithmetic mean as the surface flux for the lifting equations

$$\vec{U}^* = \frac{1}{2} (\vec{U}_L + \vec{U}_R), \quad (36)$$

and the arithmetic mean of the viscous fluxes in the numerical surface flux for the main equation

$$\vec{F}^{v*} = \frac{1}{2} \left(\vec{F}^v(\vec{U}_L, \vec{g}_L) + \vec{F}^v(\vec{U}_R, \vec{g}_R) \right). \quad (37)$$

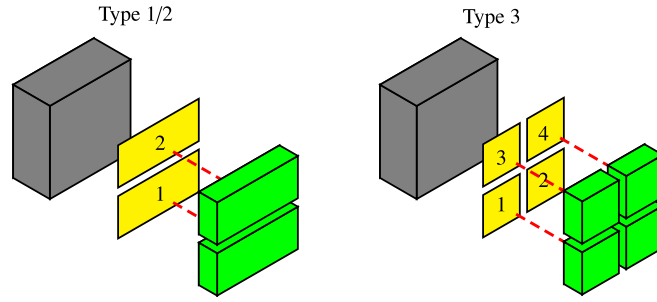


Fig. 11. Sketch of the supported mortar types. Yellow sides represent virtual small sides. Types one and two cut in different reference directions of the side.

3.2. Non-conforming meshes

Hexahedral mesh generation for arbitrary geometries in three dimensions is an area of active research and still a challenge. Many algorithms used in such situations rely on unstructured and non-conforming meshes. Thus, to expand the applicability of the solver towards more complex examples, a support for non-conforming meshes is needed. The so-called mortar technique was introduced by Kopriva [58] to handle such interfaces in a conservative and high order accurate way. We restrict ourselves to three simple types of non-conforming interfaces, which are shown in Fig. 11. For types one and two, a big side is halved into two smaller sides. The first type specifically cuts in the first computational direction on the side, type two in the second direction. A combination of the first two results in the third type, where the big side is cut into four smaller sides. It should be noted that the cuts are always made in the middle of the surface in reference space.

The main idea of the mortar technique can be summarized as follows:

- Interpolate the solution from the big side to the small sides.
- Calculate the numerical flux on the small sides.
- Project the flux onto the big side.
- Compute the surface integral for the small sides and the big sides.

The only additional steps introduced by the mortar technique are the interpolation to the small sides and the projection onto the big sides, since flux calculation and surface integral are also performed for conforming sides. In the following, the interpolation and projection operators will be derived. We only investigate the type one mortars in detail, since the other types follow in a similar fashion. To distinguish the computational coordinates used on the surface from those in the volume, we will use the notation (η^1, η^2) for the latter.

For the type one mortar, the side is cut in half along the η^1 direction, which means that the η^2 coordinate is still congruent. Thus, only one-dimensional operations are needed to interpolate the solution from the big side to the small sides, and the degrees of freedom on the small sides can be computed as

$$\vec{u}_{ij}^L = \sum_{m=0}^N \vec{u}_{mj}^B \ell_m(\eta_i^L) \quad \text{and} \quad \vec{u}_{ij}^U = \sum_{m=0}^N \vec{u}_{mj}^B \ell_m(\eta_i^U) \quad \forall i, j. \quad (38)$$

The superscripts are denoting the sides where the respective quantities are evaluated. The big side is marked with B , the two smaller sides with U and L for upper and lower. Since we are interpolating from the big side, the positions of the interpolation nodes η_i^U and η_i^L must be evaluated in the coordinate system of the big side. The interpolation procedure can be efficiently computed as

$$\vec{u}_j^U = I^L \cdot \vec{u}_j^B \quad \text{and} \quad \vec{u}_j^L = I^U \cdot \vec{u}_j^B \quad \forall j, \quad (39)$$

where the vector $\vec{u}_j$ contains all the degrees of freedom in η^1 direction at a specific position $\eta^2 = \eta_j$. The interpolation matrices are defined as

$$I_{im}^L = \ell_m(\eta_i^L) \quad \text{and} \quad I_{im}^U = \ell_m(\eta_i^U). \quad (40)$$

Once the values are interpolated to the small sides, the usual Riemann solvers can be used to compute the numerical flux functions. No adaption to the mortar technique is necessary here. The surface integral for the small side can then be performed as usual, but special care has to be taken for the big side. Following Kopriva [58], we use L^2 projection to compute the flux for the big side from the numerical fluxes of the two small sides. Thus, we are looking for a flux $\vec{f}^B$ on

the big side that satisfies for every test function ϕ :

$$\begin{aligned} \int_{-1}^1 \int_{-1}^1 \tilde{f}^B(\eta^{1B}, \eta^2) \phi \, d\eta^{1B} \, d\eta^2 &= \int_{-1}^1 \int_{-1}^1 \tilde{f}^L(\eta^{1L}, \eta^2) \phi(\eta^{1L}, \eta^2) \, d\eta^{1L} \, d\eta^2 \\ &+ \int_{-1}^1 \int_{-1}^1 \tilde{f}^U(\eta^{1U}, \eta^2) \phi(\eta^{1U}, \eta^2) \, d\eta^{1U} \, d\eta^2. \end{aligned} \quad (41)$$

As usual, we apply Gauss quadrature, insert the polynomial approximation of the flux and use the basis functions $\ell_i(\eta^{1B})\ell_j(\eta^2)$ as test functions to discretize the above equation. The results can again be expressed as matrix–vector multiplication for each position j

$$\tilde{\mathbf{f}}_j^B = P^L \cdot \tilde{\mathbf{f}}_j^L + P^U \cdot \tilde{\mathbf{f}}_j^U, \quad (42)$$

where the projection matrices

$$P_{i\alpha}^L = \ell_i\left(\frac{\eta_\alpha - 1}{2}\right) \frac{\omega_\alpha}{\omega_i} \quad \text{and} \quad P_{i\alpha}^U = \ell_i\left(\frac{\eta_\alpha + 1}{2}\right) \frac{\omega_\alpha}{\omega_i}, \quad (43)$$

are introduced.

The other types of mortars follow in a similar fashion. For type two, one simply needs to interchange the indices for the first and second computational directions. The third type can then be constructed by concatenating the two operators defined for types one and two, again showcasing the advantages of the tensor product structure.

3.3. Split DG

A possible way to achieve robustness without the need for excessive artificial damping is to construct a numerical method that is able to mimic energy or entropy estimates, that are valid in the continuous setting on a discrete level. Recently, Gassner et al. [38] have shown how to design such schemes based on the strong form DGSEM on LGL nodes. This specific version of DG possesses the diagonal norm summation-by-parts (SBP) property, a discrete analogon to integration by parts. As Fisher and Carpenter [59] have shown, the application of a differentiation operator that fulfills the SBP property can be re-written to make use of numerical volume flux functions. This allows us to express the volume integral in the strong form on LGL nodes (discarding the influence of the Jacobian for simplicity, and only for the first direction) as:

$$\sum_{\alpha=0}^N \tilde{\mathbf{F}}_{\alpha jk}^1 D_{i\alpha} \approx 2 \sum_{\alpha=0}^N \tilde{F}^{\#,1}(\tilde{\mathbf{U}}_{ijk}, \tilde{\mathbf{U}}_{\alpha jk}) D_{i\alpha}, \quad (44)$$

with the two-point flux function $\tilde{F}^{\#,1}(\tilde{\mathbf{U}}_{ijk}, \tilde{\mathbf{U}}_{\alpha jk})$. While every two-point flux function that is consistent and symmetric can be used for a valid numerical scheme, additional properties might be achieved by choosing specific flux functions. These properties include the conservation of entropy, following the definition of Tadmor [60], or preservation of kinetic energy as defined by Jameson [61]. Gassner et al. [38] also showed how to relate the two-point fluxes to the discretization of split forms of the advection terms of non-linear equations, which have been used extensively in the finite difference community to stabilize simulations (see e.g. [62,63]). Thus, a framework was created that allowed for the direct translation of many available split-formulations into the DG scheme, leading to the so-called *split DG* method.

With the introduced discretization of the differentiation operator based on volume flux functions, the semi-discrete version of the split DGSEM formulation can be written as:

$$\begin{aligned} \frac{\partial \tilde{\mathbf{U}}_{ijk}}{\partial t} = & - \left[2 \sum_{\alpha=0}^N \tilde{\mathcal{F}}^{\#,1}(\tilde{\mathbf{U}}_{ijk}, \tilde{\mathbf{U}}_{\alpha jk}) D_{i\alpha} + \left([\tilde{f}^* \hat{s} - \tilde{\mathcal{F}}^1]_{j,k}^{+\xi^1} \hat{\ell}_i(1) + [\tilde{f}^* \hat{s} - \tilde{\mathcal{F}}^1]_{j,k}^{-\xi^1} \hat{\ell}_i(-1) \right) \right. \\ & + 2 \sum_{\beta=0}^N \tilde{\mathcal{F}}^{\#,2}(\tilde{\mathbf{U}}_{ijk}, \tilde{\mathbf{U}}_{i\beta k}) D_{j\beta} + \left([\tilde{f}^* \hat{s} - \tilde{\mathcal{F}}^2]_{i,k}^{+\xi^2} \hat{\ell}_j(1) + [\tilde{f}^* \hat{s} - \tilde{\mathcal{F}}^2]_{i,k}^{-\xi^2} \hat{\ell}_j(-1) \right) \\ & \left. + 2 \sum_{\gamma=0}^N \tilde{\mathcal{F}}^{\#,3}(\tilde{\mathbf{U}}_{ijk}, \tilde{\mathbf{U}}_{ij\gamma}) D_{k\gamma} + \left([\tilde{f}^* \hat{s} - \tilde{\mathcal{F}}^3]_{i,j}^{+\xi^3} \hat{\ell}_k(1) + [\tilde{f}^* \hat{s} - \tilde{\mathcal{F}}^3]_{i,j}^{-\xi^3} \hat{\ell}_k(-1) \right) \right] \\ & \forall i, j, k. \end{aligned} \quad (45)$$

Here, the contravariant two-point fluxes are defined in the following way, for e.g. the first computational direction:

$$\begin{aligned} \vec{\mathcal{F}}^{\#,1}(\vec{\mathbf{U}}_{ijk}, \vec{\mathbf{U}}_{\alpha jk}) = & \left[\vec{F}^{\#,1}(\vec{\mathbf{U}}_{ijk}, \vec{\mathbf{U}}_{\alpha jk}) \{ \mathcal{J}a_1^1 \}_{(i,\alpha)jk} \right. \\ & + \vec{F}^{\#,2}(\vec{\mathbf{U}}_{ijk}, \vec{\mathbf{U}}_{\alpha jk}) \{ \mathcal{J}a_2^1 \}_{(i,\alpha)jk} \\ & \left. + \vec{F}^{\#,3}(\vec{\mathbf{U}}_{ijk}, \vec{\mathbf{U}}_{\alpha jk}) \{ \mathcal{J}a_3^1 \}_{(i,\alpha)jk} \right], \end{aligned} \quad (46)$$

where the arithmetic mean is denoted by $\{\cdot\}$.

For the surface fluxes, the same two-point fluxes as in the volume can be used. If those fluxes would be e.g. conservative in an entropy, then the (semi-discrete) scheme would also be entropy-conserving. Since the goal is mainly to construct a scheme that is guaranteed dissipative in the respective quantity, additional dissipation terms are added for the surface fluxes, based e.g. on Roe-type dissipation.

3.4. Shock capturing

High order methods suffer from well-known stability problems in non-smooth regions of the solution, for example across the shocks and discontinuities occurring in compressible gas dynamics. Adopting high order methods for these situations despite this inherent mismatch usually follows along two lines of thought: The non-smooth solution is regularized to bring it into the approximation space of the discretization method. This is typically achieved by the introduction of a local dissipative operator that acts on the high gradients of the solution and essentially smears them out. A classical example of this is the method of artificial viscosity, originally introduced for finite difference schemes by von Neumann and Richtmyer [64]. An adaption to high order DG schemes was described by Persson and Peraire [65]. The second general approach to shock capturing is to combine or replace the high order discretizations by more robust operators, for example WENO reconstruction [66]. Another option is to reduce the local order of the approximation, e.g. revert to first order, often in combination with h -refinement [67]. The method implemented in *FLEXI* for shock capturing follows a variant of this method. The general idea is to replace the DG operator in troubled cells by a finite volume scheme and then take advantage of the excellent shock capturing properties of this formulation. Since each DG cell contains $(N+1)^d$ degrees of freedom, the finite volume formulation can take advantage of this information richness by transferring the high order polynomial into $(N+1)^d$ first order finite volume representations. This can be understood as a reinterpretation of the DG solution on a structured finite volume mesh. This approach not only moderates the loss in effective resolution when switching from DG to FV, but also makes the implementation easy and efficient through the reliance on the same data structure. In this sense, for shock capturing, we append the DG scheme by the option of locally switching to a block structured FV scheme. Details on the method are given in the following sections, the implications for the implementation are discussed in Section 4.5.

3.4.1. Finite volume scheme on subcells

In Section 3.1, we have derived the DGSEM formulation in arbitrary dimensions. In order to keep the formulation simple and the equations manageable, we will restrict the discussion of the FV operator in the following sections to 2D. However, it will become apparent that the extension to 3D is naturally given by the tensor product structure of both the DG and FV discretizations. The discussion herein follows the more extensive descriptions in [68,69]. We thus start the discussion by recalling that in each single DG element, we approximate the solution by a tensor product of 1D Lagrange polynomials of degree N , which are associated with $N+1$ Legendre–Gauss(–Lobatto) nodes, leading to $(N+1)^2$ number of degrees of freedom $nDOF_{DG}$ in each DG element E . In theory, the number of finite volume subcells and thus $nDOF_{FV}$ per element is completely independent from $nDOF_{DG}$. However a strong reason exists to choose $nDOF_{FV} = nDOF_{DG}$: In this case, the same data structures for storing both solution representations can be used which saves memory and avoids data copying operations. It is thus a natural expression of the notion outlined above that FV and DG are a different interpretation of the same underlying solution. In addition, this choice for the number of finite volume subcells also makes sense from a load-balancing point of view: For an optimal load distribution, both DG and FV elements should incur identical computational costs. As we will show later, our choice of $nDOF_{FV} = nDOF_{DG}$ leads to nearly even computational effort for both operators. The next discretization choice concerns the distribution, size and location of the FV subcells within a DG element. A natural choice is an isotropic, equispaced distribution, leading to a regular FV grid, depicted in Fig. 12. It should be noted that with this distribution, the FV subcells inside a DG element can be interpreted as a local “block” grid. Adjacent blocks are then connected via the surface fluxes, and thus may be interpreted as a block-structured FV discretization.

Since the distribution is constructed in reference space, each FV subcell has an area of $w \cdot w$, with $w = \frac{2}{N+1}$. Each of the subcells now contains a DG solution point ij , so we label the FV cells as κ_{ij} and formulate the FV scheme of the transformed equation Eq. (6) on each of them as

$$\int_{\kappa_{ij}} \mathcal{J} \frac{\partial \vec{\mathbf{U}}}{\partial t} + \nabla_{\xi} \cdot \vec{\mathcal{F}} \, d\xi = 0, \quad \forall \kappa_{ij} \in E. \quad (47)$$

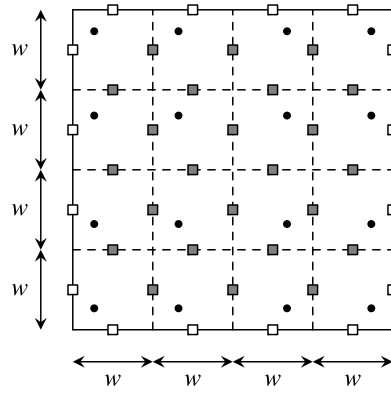


Fig. 12. DG reference element split into FV subcells --- with Gauss points • of the original DG reference element, locations of the inner ■ and the interface □ boundary fluxes as well as the sizes w of the subcells.

Applying the divergence theorem yields the classical FV formulation

$$\int_{\kappa_{ij}} \mathcal{J} \frac{\partial \vec{U}^{FV}}{\partial t} d\vec{\xi} + \int_{\partial \kappa_{ij}} \vec{F}(\vec{U}^{FV}) \cdot \mathcal{N} dS_{\kappa_{ij}} = 0, \quad \forall \kappa_{ij} \in E, \quad (48)$$

where we have also introduced the superscript FV to distinguish the DG and FV solution. We will now first derive the operators that allow consistent transformation between the DG and FV solutions, and then discretize Eq. (48). To ensure conservation, we require that

$$\int_E \vec{U} d\vec{\xi} = \int_E \vec{U}^{FV} d\vec{\xi} = \sum_{i,j=0}^N \int_{\kappa_{ij}} \vec{U}^{FV} d\vec{\xi} \quad (49)$$

holds. The above relation is discretized by inserting the polynomial representation of the DG method and the constant solution of the FV representation, exploiting the tensor product structure and applying Gauss quadrature in each subcell to obtain

$$\frac{w}{2} \sum_{k=0}^N \sum_{\lambda=0}^N \sum_{i=0}^N \vec{U}_i \ell_i(\xi_\lambda^k) \omega_\lambda = \sum_{k=0}^N \vec{U}_k^{FV} w, \quad (50)$$

relating the DOFs in the FV subcells with the DG degrees of freedom. Here, ξ_λ^k are the integration nodes in the k th subcell. With the help of the Vandermonde matrix

$$V_{DG \Rightarrow FV} := \left\{ \frac{1}{2} \sum_{\lambda=0}^N \omega_\lambda \ell_i(\xi_\lambda^k) \right\}_{k,i=0}^N, \quad (51)$$

Eq. (50) can be rewritten as

$$\vec{U}^{FV} = V_{DG \Rightarrow FV} \vec{U}, \quad (52)$$

which can be implemented as a simple matrix–vector multiplication in one spatial dimension. The backward transformation $FV \Rightarrow DG$ is facilitated by $V_{FV \Rightarrow DG} = V_{DG \Rightarrow FV}^{-1}$. Turning now to the discretization of Eq. (48), the usual steps for an FV scheme are followed, i.e. the introduction of a numerical flux function at the cell interfaces and the approximation of the integrals by the midpoint rule. With J_{ij} being the integral value of the original Jacobian determinant inside subcell κ_{ij} and $f_{i,j+\frac{1}{2}}$ for example being the flux at the top edge of the ij th subcell, the FV scheme on the block-structured grid reads

$$w J_{ij} \frac{\partial \vec{U}_{ij}^{FV}}{\partial t} = - \left[f_{i-\frac{1}{2},j} \left(\vec{U}^{FV-}, \vec{U}^{FV+}, \mathcal{N} \right) + f_{i+\frac{1}{2},j} \left(\vec{U}^{FV-}, \vec{U}^{FV-}, \mathcal{N} \right) \right] \\ - \left[f_{i,j-\frac{1}{2}} \left(\vec{U}^{FV-}, \vec{U}^{FV+}, \mathcal{N} \right) + f_{i,j+\frac{1}{2}} \left(\vec{U}^{FV-}, \vec{U}^{FV+}, \mathcal{N} \right) \right]. \quad (53)$$

The first order FV method described up to now is overly dissipative away from the shocks. To ameliorate the properties of the scheme in smooth regions adjacent to the shock, we extend it by a second order reconstruction procedure. Slope limiters are introduced to avoid an associated loss of the TVD (total variation diminishing, see e.g. [70,71]) property; we apply the reconstruction and limiting procedure to primitive variables to prevent violation of thermodynamic relations.

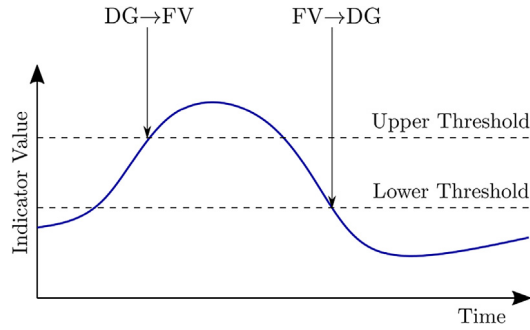


Fig. 13. Indicator based switching between DG and FV sub-cell method.

The hybrid approach leads to mixed interfaces where DG and FV elements are placed next to each other. Since both operators are coupled to the surroundings weakly via flux functions, it is natural to use this for the DG/FV interface definition as well. However, since the points of flux evaluation of the FV and DG schemes differ along their interfaces, a direct computation of the pointwise flux functions is not possible. Instead, the DG solution $\bar{U}_j^+$ is first evaluated at the FV solution points with the help of the Vandermonde matrix defined in Eq. (51). The only difference to Eq. (52) is that here it is applied to face data only and not to the whole volume. Once this operation yields $\bar{U}_j^{FV+}$, the pointwise numerical flux function is then evaluated. It can directly be applied to the DOF in the FV element to update the solution in subcells. The transfer to the DG solution points occurs by the inverse of Eq. (51). Note that a FV/DG interface can also occur across the mortar interfaces discussed in Section 3.2. While in principle the coupling in this case uses the same concepts that are presented here, some subtleties and special cases need to be considered. For a detailed discussion, we refer the reader to [72].

3.4.2. Troubled cell indicators

In the previous sections, we have presented the operators that allow a fast and efficient switching of the element type from a DG to a FV-subcell element and vice versa. It is clear that the switching itself costs some computational effort and should occur as infrequently as possible. Also, cells that require FV treatment to ensure stability should be detected and switched in time, while they should be able to revert back to the DG representation if the solution permits. Detecting candidate cells for switching thus requires the evaluation of some criterion – either in an a priori or an a posteriori approach. In the latter, less frequently used case, all elements are computed at each time step with the DG scheme and the solution is evaluated afterwards. If it shows signs of instability, the time step is repeated, with the marked cells being treated by a more stable method [73]. In the more prevalent a priori method, the solution at the beginning of each time step is examined, and cells are then flagged accordingly and treated by the appropriate numerical scheme. In any case, evaluating the cells is achieved by some form of indicator function. A large number of indicators exist which are based on physical reasoning, jump conditions or model solution representation [65,74,75]. All of these cited indicators are available in *FLEXI*. They are implemented to return a large value in regions of non-smoothness and a low value otherwise. Starting from purely DG elements, if the indicator increases over a given threshold, the element in question is switched to a FV subcell element before the next time step. It remains in this discretization state until the indicator drops below a given value. The solution in this element is then seemed smooth enough to warrant a DG discretization of residual without the introducing stability issues. Numerical studies have shown that a single threshold might lead to excessive back- and forth switching and a high sensitivity to the actual, user-defined value. Therefore, the single threshold has been replaced by an upper and a lower threshold as visualized by the range in Fig. 13.

3.5. Time integration

While the semi-discrete systems can be, in principle, advanced in time by any suitable time integration method, both implicit and explicit, we restrict ourselves to explicit Runge–Kutta type schemes, specifically low-storage versions. For this class of methods, there is no need to store the result from every Runge–Kutta stage, thus reducing the memory requirements. In general, the schemes used are based on the two-register formalism introduced by Williamson [76], and several specific schemes are available to choose from. In the hybrid DG/FV approach, both systems are advanced in time with the same method.

Time step restrictions

Since we use explicit Runge–Kutta time integration, the time step is limited by the famous CFL condition [77] for a given mesh size. For our DGSEM, the condition can be written as:

$$\Delta t \leq \text{CFL} \cdot \gamma_{RK}^1(N) \frac{\Delta x}{\lambda^c (2N + 1)}, \quad (54)$$

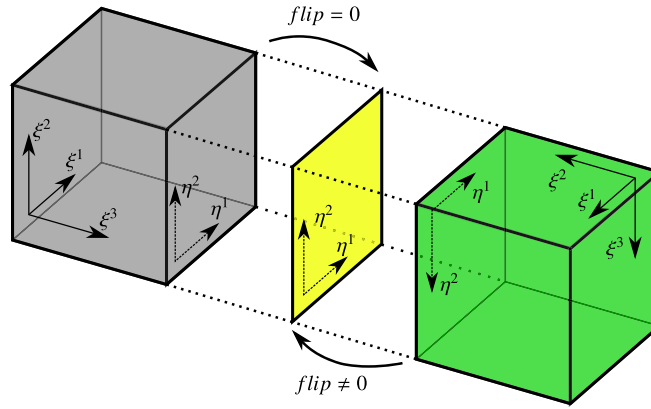


Fig. 14. Schematic of relationship between volume and side coordinate systems. Volume systems denoted by solid coordinate axis, side systems by dotted axis. The gray element is the master for the yellow side, the green element the slave. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

where CFL is the CFL number, Δx the size of an element and λ^c the largest convective signal velocity. Usually, a scaling factor of $\frac{1}{2N+1}$ is included to take the high order solution in the element into account [78], while other authors use different scaling factors, see e.g. [79]. In practice, this scaling is not exact, and the user might not be able to tell the valid range of the CFL number for each configuration. To circumvent this problem, we introduce an additional correction factor $\gamma_{RK}^1(N)$, that depends on the polynomial degree and the set of interpolation nodes. This factor is derived from numerical experiments such that the usual condition $CFL \leq 1$ must hold to obtain a (linear) stable time step.

The largest signal velocity λ^c depends on the specific set of equations currently considered, where those velocities are given by the eigenvalues of the system matrix. While this restriction stems from the hyperbolic part of the equations, the parabolic fluxes also introduce time step limitations. Those can be expressed analogously to the hyperbolic restriction

$$\Delta t \leq CFL_d \cdot \gamma_{RK}^2(N) \frac{\Delta x^2}{\lambda^v (2N+1)}, \quad (55)$$

where a diffusive CFL number CFL_d was introduced. The factor λ^v here is the largest eigenvalue of the diffusion matrix. Since this restriction scales as Δx^2 , it is especially restrictive in regions where the mesh size reduces significantly.

Both of these conditions must be satisfied for a stable simulation, so that the minimum of the two will be used for the time step in the computation. The conditions are evaluated for each of the directions in the reference element, thus the physical size of the element enters through the transformations.

For cells that are discretized with the FV method, the time step restriction can be expressed as

$$\Delta t \leq CFL \cdot \gamma_{RK}^1(0) \frac{\Delta x_{FV}}{\lambda^c}, \quad (56)$$

where $\Delta x_{FV} = \Delta x_{DG}/(N+1)$. Note that for a given flow field the time step given by the DG representation is smaller than for the FV subcell representation.

4. Implementation details

This section will describe how the DGSEM and the hybrid DG/FV scheme is implemented in the solver. Aspects that will be discussed include the data structures, parallelization approach and general structure of the algorithm.

4.1. Coordinate systems for unstructured grids

Operations in the DG method either act on the volume of a cell or on sides, which are shared by two adjacent elements (excluding the boundaries). In our mesh structure, elements and sides are thus handled separately, and sides are only stored once and connected to the respective elements through a mapping. To be able to handle unstructured grids, several different coordinate systems and their relationship must first be introduced. Fig. 14 shows a sketch of the used systems. Each element has a *local volume coordinate system*, denoted by ξ^1 , ξ^2 and ξ^3 , which is always a right-hand system. On each side of the element, a *local side coordinate system* (η^1 and η^2) is introduced, following the definitions given by the CGNS standard, see e.g. [80]. But, as each side is shared by two elements, local side coordinate systems from those two elements might differ. To obtain a definitive system that should be used for operations on that side, one of the two elements will be designated as the *master* of that side (and the other as *slave*). The local coordinate system of the master element will then be used as the *global side coordinate system* for that respective surface. To convert from local side systems to global

ones, a *flip* is defined that represents how the two systems are oriented with respect to each other. A flip of 0 is used to indicate a master side.

The conversion between side and volume information thus requires first a mapping between the volume coordinate system and local side systems. This only depends on the currently considered local side (e.g. $+\xi^1$) and the relation can be implemented as a lookup-table. Second, the mapping between local and global sides is uniquely defined by the flip, and since only a limited number of possible combinations exist, this can also be computed and stored before the actual simulation starts.

4.2. Domain decomposition

The natural parallelization approach for the DG method is to divide the elements in the mesh into separate domains (each containing one or more mesh elements), and to assign each domain to an available processing unit. We want the decomposition to be flexible, so it can handle computations on an arbitrary number of processes without the need for expensive re-calculations. Additionally, domains should be compact and contiguous, to reduce the required amount of communication between the domains. As has been mentioned in Section 2.1.1, *HOPR* provides the SFC sorting to achieve this. When a parallel computation is started, the SFC can simply be subdivided into as many parts as processing units are available.

4.3. Parallelization aspects

FLEXI uses a distributed memory approach for parallelization. Thus, across boundaries of the mesh domains, communication will be necessary. An advantage of the DG method is that, due to the local approximation space and the absence of continuity requirements on the solution across element interfaces, only surface data needs to be communicated to the direct neighbor elements, reducing the amount of data compared with e.g. high order finite volume schemes significantly. Thus, all communication operations are based on side data structures. The introduction of global arrays for the side based data compromises the data locality, but is preferred due to ease of implementation in the chosen communication pattern. To perform the actual communication, the MPI protocol is used. It is widely available and optimized on HPC architecture, and allows for non-blocking operations that can be used to hide communication time. For efficient communication hiding, one can take advantage of the high density of local operations of the DG operator, as is explained in detail later.

The communication patterns are built at the start of the simulation, as each processor can determine from the SFC with which other processors it needs to communicate. Only very few operations require global communication, notably the identification of a global smallest time step for the Runge–Kutta time integration scheme.

At MPI boundaries, the distinction between master and slave elements of a side will also influence the amount of work that the MPI domain needs to do, since only the domain with the master element will compute the numerical flux function for that surface. Thus, to ensure optimal load-balancing, we try to evenly distribute slave and master sides at MPI boundaries for each domain.

4.4. Algorithm sequence

Fig. 15 (ignoring the parts given in red) shows an overview of the algorithmic implementation of the DG operator. We start by considering the DOFs in the volume U obtained from initialization or during the last time step. Since the computation of the numerical fluxes requires the solution on the surface of the elements, as a first step we need to extrapolate the volume solution to the sides of each element. Master and slave elements of the respective side will use different arrays to store both values of the double-valued solution. If LGL nodes are used, the extrapolation reduces to a simple memory copy, since the surface nodes are included in this set of nodes.

As the numerical fluxes are unique on a side, they only need to be computed once. We arbitrarily decide that the domain that owns the master element of that side will compute the flux function. In a parallel setting, this means that now the MPI threads owning slave sides on the MPI boundaries must communicate the extrapolated solution to the threads owning the respective master sides. We employ non-blocking communication, thus the communication is initiated, but other computations can be commenced while the actual data transfer happens. Ignoring the lifting procedure (gray box in Fig. 15) for the moment, the volume integral of the DG operator can now be computed. This is a purely cell-local operation, and can thus be done while the communication of the surface data is still ongoing. The large amount of cell-local work enables effective hiding of the communication in the DG operator.

Once the volume integral is finished, the communication must be finished. As a next step, the master sides calculate the numerical flux from the double-valued surface states. This process can be split in two parts, where first only the surfaces that belong to MPI boundaries are processed. This allows us to immediately start the communication of the fluxes back to the slave cells, and then use the computation of the fluxes at the inner cell boundaries to again hide that communication. One step further, the actual computation of the surface integral over the numerical fluxes starts with the inner sides. Only when all inner sides are finished, the communication of the fluxes must be closed, and the surface integral over the remaining sides can be computed.

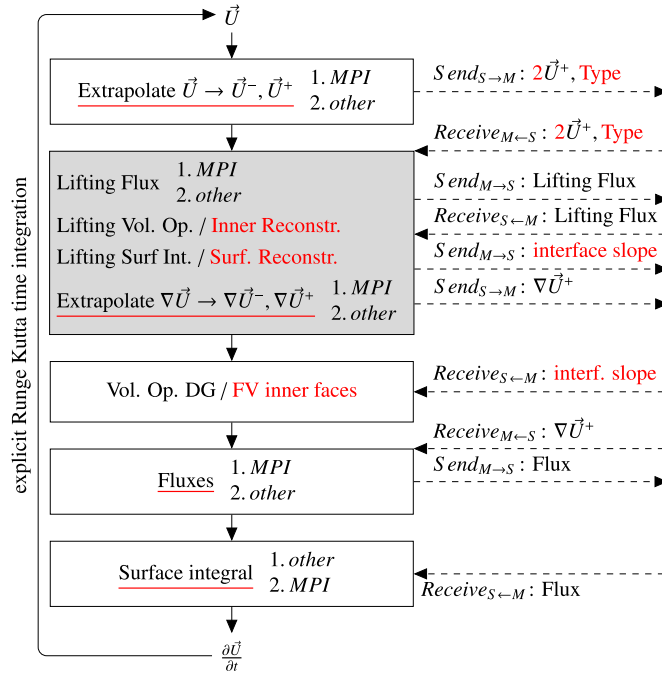


Fig. 15. Flow chart of the hybrid discontinuous Galerkin/finite volume subcells operator: Procedures underlined in red are modified to perform their specific task either for DG or FV elements. At the same time the lifting of the DG elements is computed, the 2nd order reconstruction of the FV subcells is built. The counterpart to the DG volume integral is the fluxes over inner FV subcell interfaces. Additional communication is required. Besides the information of which type (DG or FV) an element at a MPI interfaces is, for the FV reconstruction a second array of face data has to be transmitted, which is indicated by the red numbers 2. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Once these steps are completed, the calculation of the semi-discrete DG operator is finished. The result then enters the low-storage time integration scheme, and the operator evaluation starts again at the next stage.

If gradients are required for the considered equation system, the lifting procedure basically introduces another layer of a similarly functioning operator. The lifting is again split into the volume and the surface contribution, and communication hiding is performed in the same manner.

4.5. FV shock capturing

In this section, we present the implementation of the hybrid DG/FV method. A main design principle was to keep the general structure of the DG operator intact, and to find equivalents for each of the components in the FV operator. In combination with the fact that we keep the same data structure for the solution in the hybrid scheme, this allows for an implementation that is only minimally intrusive for the DG operator.

Fig. 15 also includes the extension towards the hybrid DG/FV method, with the parts that require different treatment marked in red. For FV cells, the extrapolation of the solution to the boundaries changes to a simple copy of the cell mean value at the element interface to the respective arrays, such that this information can be used to compute a slope, and is communicated to the master element in a parallel setting. Additionally, to prepare for later reconstruction, the slope between the first and the second subcell away from the interface is computed and also communicated. We need to do this since, as for the pure DG case, only the master side will compute the flux functions. For this, it needs to know the limited slope from the slave element, which is a function of the slope across the interface itself and the one between the next two subcells (the one we communicate). If a DG element is considered, instead we communicate the solution at the first Gauss point. This means we need to send and receive twice the amount of data as in the DG case, both the solution and the values needed for the reconstruction. We also communicate a single integer indicating the type of the cell (DG or FV), such that the algorithm can decide what type of interface it needs to handle.

The equivalent for the lifting procedure in DG is the computation (and limitation) of the slopes between the subcells for the FV elements. While there is no real volume work for the FV scheme, we perform the slope computation across the inner subcell interfaces at the same time as the volume integral is computed for the DG scheme. The reason is that both of those require no communication between MPI domains, and can thus be used to hide the communication latency. While the surface integral is done for the DG method, the remaining slopes across the cell-interfaces can be computed on the master sides of the elements. Here, the limiting requires the additional data that was communicated beforehand.

The limited slopes at the cell interface need to be send back from the master element to the slave sides, since they are needed to limit the slopes for those elements.

With the lifting/computation of the slopes complete, the volume integral of the DG operator is performed. For FV elements, here the fluxes at the inner subcell interfaces are computed and directly applied, since again those require no communication. The reconstruction of the values at the subcell interfaces from the limited slopes is also done here. In the next step the fluxes at the cell interfaces are computed. In the case of a mixed interface, the DG solution must first be transformed into the FV representation. The last step is the surface integral, where for DG cells at mixed interfaces the flux is projected back to the DG representation, and the computed fluxes are simply applied to the FV subcells at element boundaries.

While the arrays containing the solution are shared between the two schemes, other additional quantities must be stored for the hybrid scheme. This includes the metric terms in the FV representation of a cell, geometric information (normal and tangential vectors, surface elements) for inner faces as well as the physical distances between subcells in each direction to perform the reconstruction. This additional memory consumption has some impact on the performance of the scheme, and will be discussed in the next section.

4.6. Strong scaling behavior

In this section, we will investigate the strong scaling behavior of both the pure DGSEM and the hybrid DG/FV method developed for shock capturing. All the subsequent tests were performed on the Cray XC40 system *Hazel Hen* at the HLRS, which is equipped with compute nodes that consist of two Intel Xeon E5-2680 CPUs (12 cores each) and 128 GB of memory. The communication between nodes is realized using the Cray Aries high-speed interconnect. We ran a simple simulation of a free-stream on cubic Cartesian domain, with a varying number of mesh elements. The smallest mesh consisted of $[6 \times 6 \times 6]$ elements, and bigger meshes were created by doubling the number of elements individually for each direction. So the next mesh would have $[12 \times 6 \times 6]$ elements, then $[12 \times 12 \times 6]$ and so on until $[96 \times 96 \times 48]$ elements were reached. The simulations were all first executed on a single node, except for the largest mesh, where two nodes were needed to provide enough memory for that case. The number of nodes was then repeatedly doubled, until for each case only 9 elements per core remained. Thus, the minimum number of cores used was 24, and the largest cases were run on up to 49.152 cores. All simulations were advanced for 100 times steps, and each configuration was repeated five times to gain information about variance in the results. To reduce the possible amount of combinations that could be checked, we restrict ourselves to the Navier–Stokes equation system, $N = 6$ and choose the LG nodes. For the mixed DG/FV scheme, one half of the computational domain was always set to be only DG, the other to be pure FV. This is not the worst-case scenario since the amount of mixed DG/FV interfaces is limited, but a realistic one considering that regions with shocks should be compact and not spread out.

The main metric analyzed is the performance index PID, which is the time required to advance a single DOF from one stage of the Runge–Kutta time integration scheme to the next. It is computed as

$$PID = \frac{\text{wall-clock-time} \cdot \text{\#cores}}{\text{\#DOF} \cdot \text{\#time steps} \cdot \text{\#RK-stages}}. \quad (57)$$

Fig. 16 shows the results from the strong scaling tests. In the top row, we compare the parallel efficiency, which is defined as the ratio of the PID using a certain amount of nodes to the median of the PID (calculated from the repeated runs of the same configuration) using the smallest amount of nodes. Focusing on the results obtained from the DG calculations, we observe that all cases exhibit super-linear scaling, where the efficiency is larger than 100% in certain areas. This can be explained by the high memory consumption for calculations on a small number of nodes. When the number of compute nodes increases, the required memory per core decreases, and more of the used data can be stored in the fast cache of the CPU, increasing performance. This also explains why for the larger cases the super-linear behavior emerges at a larger number of cores. If the number of nodes is increased further, a drop in efficiency can be observed for nearly all cases. In this region, the remaining core-local work load might no longer be enough to hide the communication effort, which ultimately outweighs the increased cache-efficiency. The performance in this region is thus bound by latency of the underlying network. For all considered cases, the efficiency does not drop below 70%.

For the hybrid DG/FV approach, a similar picture emerges, but the super-linear behavior is not as pronounced or not present at all for the large cases. This is consistent with the fact that the FV operator increases the memory consumption of the overall method. Thus the described cache-effects are shifted towards an increased number of compute nodes, and might not appear at all since the latency is already the dominant factor in the cache-friendly regions. While the parallel efficiency might not reach the same level as the pure DG operator, still the overall behavior is deemed as sufficient.

The results for the hybrid DG/FV approach show a very similar behavior, but lack the clear optimum for the load per core that was observed for the pure DG operator. The reason for that can be found in the increased memory consumption of the FV method due to additionally stored quantities, e.g. metric terms for the FV representation. It is important to compare the absolute values for the performance at not just relative scaling, as we do here. This reveals that the performance for the hybrid method is about 15–20% worse than for the pure DG method, which is the price we pay for the hybrid operator structure.

Of course the scaling behavior is only one metric for parallel performance, and serial performance is just as important. For this reason, the absolute performance values are reported in the bottom row of Fig. 16. To judge the performance in

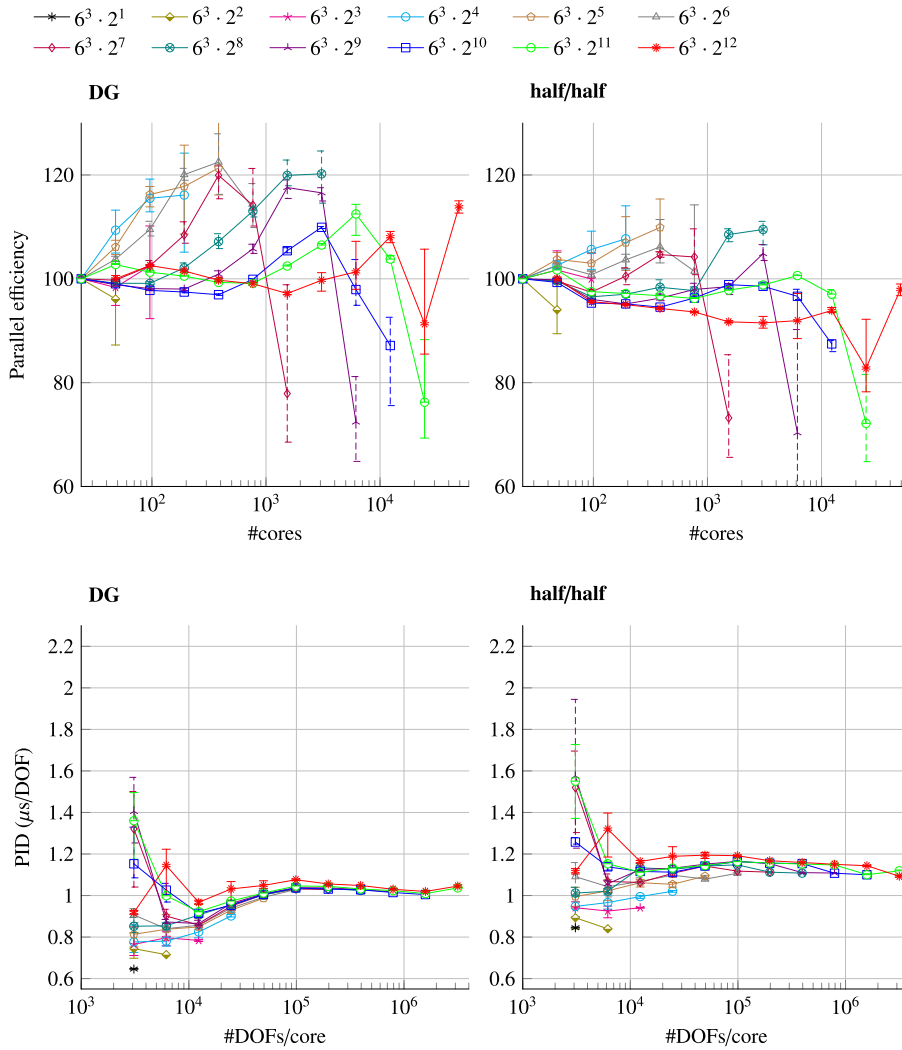


Fig. 16. Parallel efficiency and performance for the strong scaling test case. The line color/marker combination refers to a specific mesh, the number of elements is given in the legend. Each test case was repeated five times, shown are the median values, error bars indicate the smallest and biggest values observed. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the saturated area ($PID \approx 1 \cdot 10^{-6}$ s), we can compare to data published by Fehn et al. [81], who report results for a similar setup on comparable hardware, but with a different formulation of the DG method and treatment of the viscous fluxes. They report $PID \approx 3.4 \cdot 10^{-7}$ s, i.e. smaller by a factor of about three. The analysis of possible bottlenecks in the algorithm, found e.g. in the limits of memory bandwidth or latency, merits further research. Additionally, the optimization of the different parts of the algorithm is ongoing research, see e.g. the *single instruction, multiple data* (SIMD) based optimization of tensor product operations given in [82]. An important aspect among others is definitely the choice of lifting procedure, as the BR1 scheme is responsible for $\approx 40\%$ of the total compute time in a typical run.

4.7. Implementation of post-processing

As has been mentioned in the introduction, we implement a custom visualization toolchain, which is fully parallel and takes advantage of the high order nature of our solution. This toolchain combines our specifically built *POSTI* visu tool, which handles the actual computations, and the open-source software ParaView.

Fig. 17 presents the different parts of the toolchain and how they interact with each other. On the top level, ParaView provides the user interface as well as access to a multitude of visualization options and filters. An important aspect is also that ParaView is fully parallelized, allowing us to run the post-processing on a number of cores that are adequate for the problem at hand. The second layer of the toolchain is a plugin for ParaView that provides a so-called reader (essentially

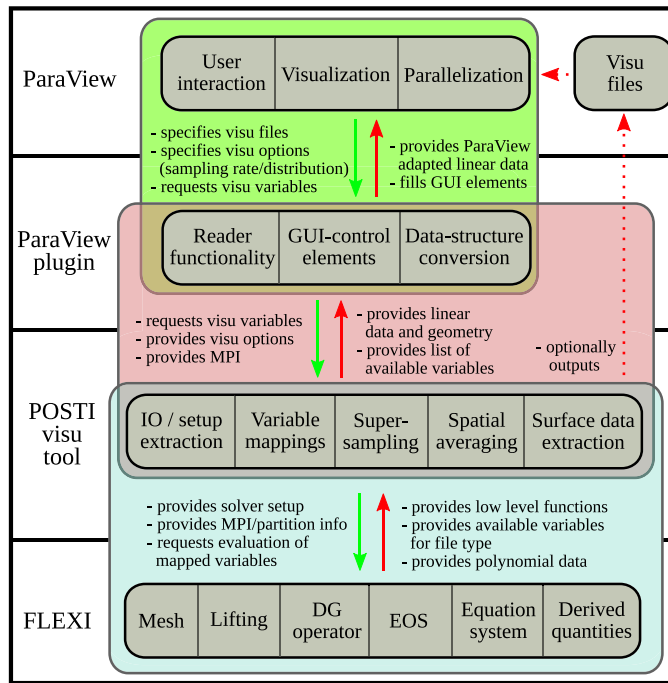


Fig. 17. Schematic of the implementation of the visualization tool.

an import functionality) for the files written by *FLEXI*. A reader defines GUI elements in ParaView, that can be used to specify options which can then be passed on to the *POSTI* program.

The plugin acts as an interface between ParaView (written in C/C++) and the custom *POSTI* tool (written in Fortran). It will pass the MPI communicator created by ParaView down to that tool, which means both of the visualization and the data processing part of the toolchain use the same parallelization. The options specified by the user in the GUI are also passed down, as a simple text file which can be parsed by the *POSTI* tool. Depending on the options selected by the user, the tool can now perform calculations. To this end, it can directly use routines originally implemented for the solver itself. This includes e.g. I/O routines, initialization of the mesh, partitioning in MPI regions, calculation of gradients using the lifting operator or the DG operator itself. Depending on the used equation system, derived quantities can be computed from the conservative variables stored in the state files. Those can range from simple calculations of primitive quantities like the temperature (taking the current equation of state into account) to complex vortex identification criterion like λ_2 , requiring knowledge of the derivatives and the calculation of eigenvalues of a matrix. The advantages of doing these calculations directly at the lowest level instead of in the visualization software are (a) the direct accessibility of all necessary data and (b) an performance advantage, since only the derived quantity needs to be passed to the visualization software, not all variables needed to calculate it. Additional features are also provided by the *POSTI* tool, for example the ability to average three-dimensional data along a regular direction, or the interpolation of data to the surfaces.

Once the variables specified by the user have been calculated, the *POSTI* tool converts them from the high order polynomial representation used internally to the linear format needed by ParaView. To reduce the errors introduced by the linear interpolation, usually a super-sampling of the high order polynomial is used. The data is passed back to the plugin, which provides it in the way needed by ParaView, which is then finally able to display the requested data. All the computation and conversion steps are hidden from the users, they simply interact with the ParaView GUI.

As will be discussed in more detail in Section 5.2, the state files also contain all parameters that were used to run the simulation next to the solution variables. Besides the aspect of reproducibility, this also means we can use that information in the post-processing. This frees the user from having to specify the details of the simulation that are necessary for the visualization process, e.g. the polynomial degree, node type or equation system used in the simulation.

5. Aspects of usability and reproducibility

While a lot of emphasis is typically placed on stringent description of numerical algorithms and mathematically precise analysis of the schemes encapsulated in open-source CFD codes, ease of access and use as well as reproducibility are often treated less thoroughly. However, conveying the expert knowledge that goes into the conceptual design of the scheme and code to the final user is important to ensure correct and reproducible results. In this section, we present

some of the concepts and methods used in the *FLEXI* ecosystem to help make the code more user-friendly and thus error-resistant and to support good reproducibility practices. It is obvious that only a framework that ensures *reproducible* results should have its place in research and development in the long run. Open-source software supports reproducibility on a conceptual level, as it lays the basic algorithms open. However, this is just a single link in a complex, interwoven chain that drives possible sources of uncertainty in the simulation stack and thus decreases reproducibility. Other, equally important aspects that drive systemic opacity are for example the user-defined choice of discretization parameters, the hardware and associated operating system, the dependence on third party libraries and the build or compile options for all the code pieces involved [83]. According to Peng [84], the gold standard in reproducibility entails not only providing all this information, but also the binaries and even the produced simulation data itself. Taking the heterogeneous and varied hardware landscape (and again, the associated first level software like drivers and operating systems) into account, perfect reproducibility would in the limit entail even hardware specifications. Clearly, already the previous level demanding compiled and linked binaries as well as full simulation data is not practical, in particular in a HPC context, where the data sets are voluminous and the user has neither full knowledge nor control over the hardware and available libraries. Despite all these hindrances, striving for a maximum amount of reproducibility increases the trustworthiness of the results, stability of the code during its lifetime and the usability of the CFD software. In the *FLEXI* framework, a number of measures supporting these notions have been included, two of which will be represented here briefly.

5.1. Regression testing

Regression tests are mainly used to ensure code stability during the development process and to uncover introduced bugs. Additionally, they provide a benchmark for code efficiency, i.e. new features should not slow down execution speed noticeably. The tests themselves are implemented in various forms. *Unit tests* are run each time the code base is compiled, and they test several core functionalities of the DG operator, e.g. computing a surface integral for a flux vector. The success of those tests is determined by comparing them with pre-calculated results. They make sure that the very basic building blocks of the framework continue to operate as expected.

Another form of tests is performed using the continuous integration capabilities of the gitlab code management framework. Each time a new commit is published, several *check-in tests* are executed. Those are computationally cheap tests, that check some basic functionalities at a higher level than the unit tests: Can the code be compiled with the most common options? Can the code be executed and run minimalistic examples like a constant state solution on Cartesian grids? More thorough tests are performed as *daily* and *weekly regression tests*. In those, most possible combinations of compile options are built and tested in various examples, aimed at testing features such as mortar interfaces, boundary conditions, Riemann solvers and time integration methods. The custom python tool *REGGIE*³ is used to compile, run and analyze those tests, which provides advanced features such as the possibility to directly calculate convergence rates from subsequent runs and compare those to expected values. The weekly tests consist of real-world applications, such as the flow around an airfoil or a turbulent channel flow, and check the complex interaction of multiple features at the same time. Overall, this feature thus ensures reproducibility on a programming level.

5.2. The userblock concept

One level above the specific programming or implementation details lies the management of code and simulation meta-data. These meta-data contain information about the specific code variants, the compile options and the simulation settings. While open-source software generally is seen as a step towards reducing systemic opacity, it comes with the additional challenge of managing (even more) meta-data. *FLEXI* and its supporting software can be distributed freely, modified at will and compiled on a wide range of platforms, leading to a growing number of variants. Hence, identifying and obtaining the underlying code version for reproducing results can be at least time consuming if not impossible. In order to assist in replicating a simulation setup at a later date, *FLEXI* implements a robust mechanism to re-obtain source code, compiled binaries and simulation parameters from each solution file written by *FLEXI* with minimal storage overhead.

The flow solution at a given time t is stored in a HDF5 format. To each of these so called “state” files, a block of custom meta-data is appended, subsequently called the “user-block”. This user-block contains all the available information at run and compile time to ensure that (a) a state file can be associated with a specific simulation run in case this information becomes inadvertently lost and (b) the simulation can be recreated with the same code variant and user settings. The user-block format features a simple syntax with sub-groups. It can contain both text and binary data, as well as compressed data. A schematic of the data in the user-block is depicted in Fig. 18. The first block provides an exact copy of the settings file with the simulation parameters. A second compressed block contains data from an object file, which is directly generated at compile-time and linked statically into the solver executable. It contains sufficient information to rebuild the executable. Code version control is based on Git [85], so we store the repository that was used to clone the current code base, the git commit and any local modifications to the code which have not been uploaded to the repository in the form of file patches, which can be applied to recover the local code variant. Compile-time flags and

³ <https://github.com/reggie-framework/reggie2.0/>

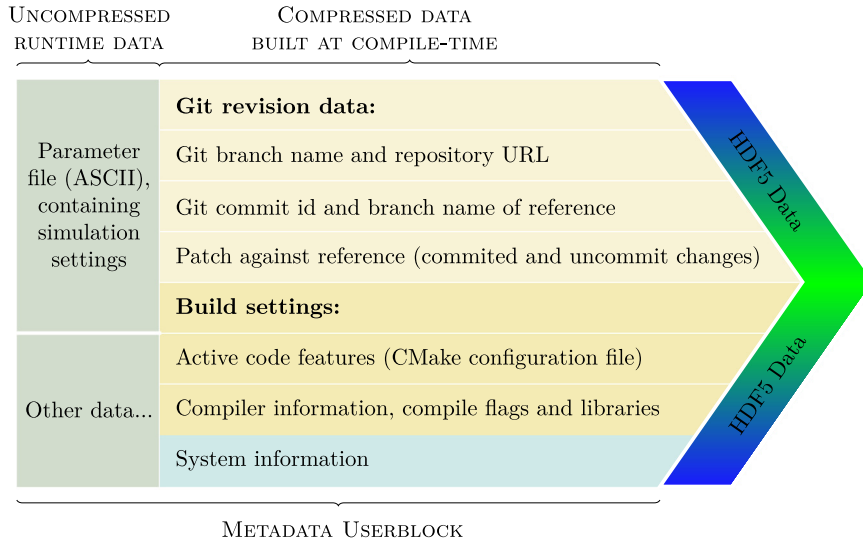


Fig. 18. Format description of meta-data in the user-block section of the solution files.

options in the form of preprocessor flags that are generated and interpreted by the CMake build system are also stored. Access to the information contained in the user-block is facilitated by scripts that are part of the *FLEXI* repository. Directly rebuilding the *FLEXI* variant and restoring the configuration can also be initiated. This process consists of downloading the code from the specified repository, checking out the correct revision, applying the patches and rebuilding the binaries using the stored CMake configuration. Besides serving as a quick and robust mechanism to help with reproducibility, the user-block data can also be queried by other parts of the *FLEXI* ecosystem. For example, the post-processing tool chain presented in Sections 2.3 and 4.7 can recover the information stored therein and incorporate it consistently into the post-processing of the simulation data.

5.3. Documentation and tutorials

The documentation for *FLEXI* is available from the project website. Besides a description of the building process and the features of *FLEXI*, it also contains a number of tutorials that introduce the basic functionalities and give a starter for code modifications. For code developers, a code documentation based on Doxygen is also available from the project website. It is rebuilt automatically to reflect updates to the master branch.

6. Applications

In this section, we briefly summarize three challenging simulations conducted with *FLEXI*, which highlight some of the features presented in the previous chapters. As a first example, we discuss a wall-resolved LES of a NACA 64418 airfoil at high Reynolds number. This simulation combines some of the challenges addressed before, as it requires $\mathcal{O}(10^8)$ DOF per solution variable, a surface- and volume-curved mesh, a stable scheme for implicitly modeled LES and an pre- and post-processing framework capable of handling the large-scale data seamlessly. The second example, a shock-vortex interaction, demonstrates the ability of the hybrid DG/FV method to capture shocks and retain the advantageous high order properties in regions of smooth flow. Our last example highlights a HO zonal LES approach for a highly sensitive acoustic feedback mechanisms. It features a wall-model as well as a synthetic turbulent inflow, which both do not generate noticeable acoustic disturbances. A comparison against an experimental measurement campaign shows that *FLEXI* is capable of capturing the intricate interactions between hydrodynamics and acoustics accurately. Besides these highlighted examples here, *FLEXI* has been both extensively validated against canonical test cases as well as applied to a variety of problems, see e.g. [36,39,41,47,86–90].

6.1. LES of airfoil at $Re = 10^6$

The first example is used to demonstrate the applicability of *FLEXI* to large-scale problems in the field of aerodynamics. We compute the flow around a NACA 64418 airfoil at a chord Reynolds-number $Re = \frac{U_\infty c}{\nu} = 10^6$ and a free-stream Mach number of $Ma = \frac{U_\infty}{a} = 0.2$. Here, U_∞ is the free-stream velocity, c the chord length, ν the kinematic viscosity and a the speed of sound. The angle of attack is set to $\alpha = 4^\circ$, which will lead to a region of separated flow at the trailing edge of

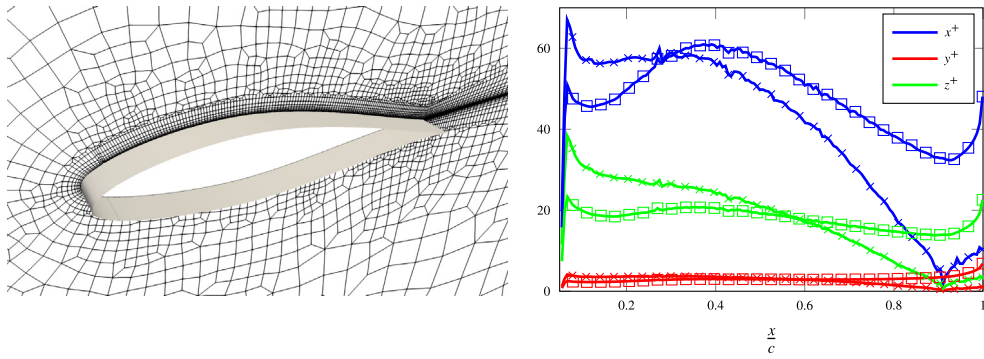
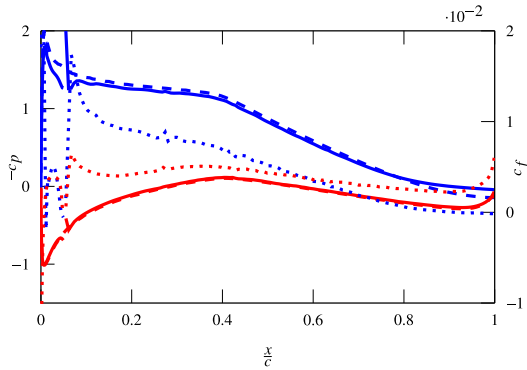
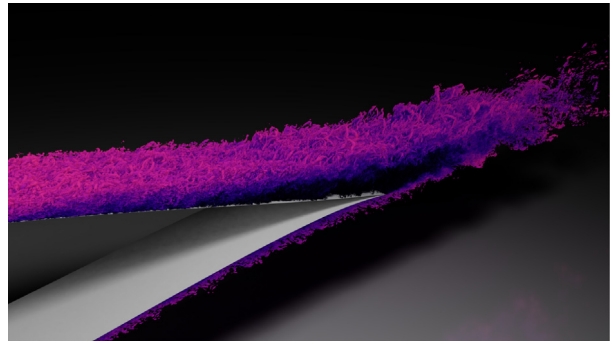


Fig. 19. Mesh used for the NACA 64418 airfoil simulation (left) and associated non-dimensional grid spacing (right). Crosses denote the suction side, squares the pressure side.



(a) Comparison of pressure and friction coefficient (c_p and c_f) with results from XFOIL. Blue lines represent results from the suction side, red lines on the pressure side. (—) c_p FLEXI, (---) c_p XFOIL, (.....) c_f FLEXI.



(b) Isocontour of Q-criterion, colored with velocity magnitude. Close-up of trailing edge with separated flow region.

Fig. 20. Selected results from the NACA 64418 simulation.

the airfoil. The separation process is sensitive to the details of the flow and underlying modeling. Thus an accurate and scale-resolving simulation methodology is necessary to reliably predict such phenomena.

Fig. 19 shows details of the used mesh. A C-grid of height $0.04c$ is employed in the close vicinity of the airfoil, to guarantee optimal grid quality for the turbulent boundary layer. Outside of that structured layer, the grid becomes unstructured and the mesh size is increased rapidly to minimize the total number of grid cells. A geometrical trip in the form of a protruding step of height $0.001c$ is incorporated in the mesh at $x = 0.05c$, which reliably triggers transition to turbulence at this specific location. The spanwise extend is chosen as 10% of the chord length, and the outer boundary is at least $50c$ away from the airfoil surface. The mesh spacing is chosen such that certain requirements for wall-resolved LES are met. The left plot of Fig. 19 shows the grid spacing in wall-units in the region after the turbulent trip. Since one grid cell contains several degrees of freedom per direction, the spacing is normalized by the factor $(N + 1)$. Across the airfoil, the values for x^+ range between 40 and 60, z^+ between 20 and 30 and $y^+ \approx 3$. The mesh consists in total of 229,620 elements, and is represented by polynomials of fifth order. We used ANSA[®] to generate the grid, and the surface elements were curved by splitting them as described in Section 2.1.2. The volume mesh close to the airfoil was then curved using the RBF approach, see Section 2.1.3.

The simulation was run as an implicit LES, using the methodology investigated by Flad and Gassner in [40]. It is based on the split DG formulation with kinetic energy preserving fluxes, specifically the flux introduced by Pirozzoli [91]. The equations were discretized with an eight-order scheme, leading to approximately 117 Mio. DOFs per solution variable. The simulation was advanced for 15 non-dimensional time units $T^* = \frac{u_\infty}{c}$, and the last five time units were used to gather statistics. Selected results are shown in Fig. 20. The left plot compares the pressure coefficient on the suction and pressure side with results from XFOIL [92], and they show excellent agreement, except for a small region at the trailing edge of the suction side. There, the flow separates, which cannot be predicted by the low-fidelity software XFOIL. The separation bubble is also evident in the negative wall friction coefficient, as seen in Fig. 20(a). In the right plot, the turbulent structures at the

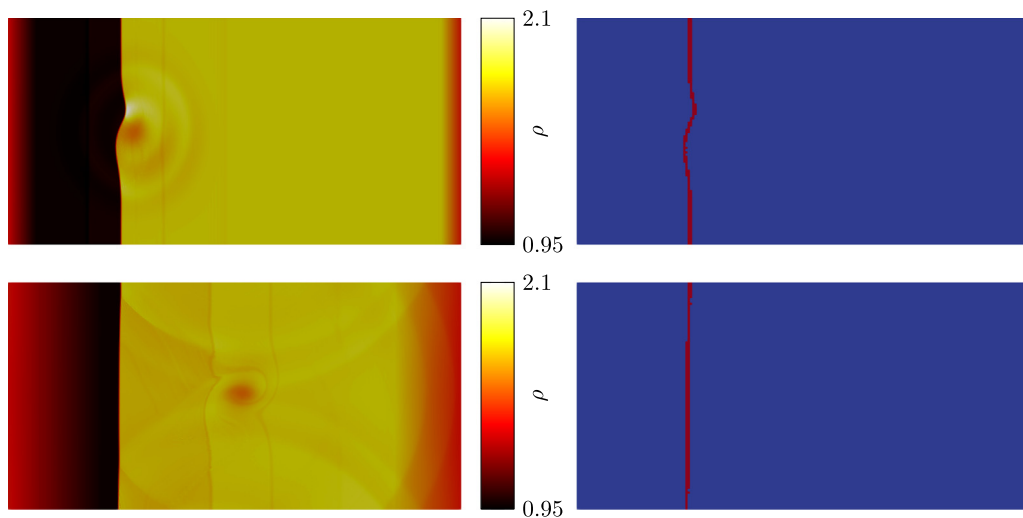


Fig. 21. Results of a shock-vortex interaction at $Ma = 1.5$. Top row: $t = 0.2$, bottom row: $t = 0.7$. The left plots visualize the density ρ , the right plots the distribution of FV (red) and DG (blue) cells. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

trailing edge are visualized in a rendering of an isocontour of the Q-criterion, colored by the velocity magnitude. The large area of low-velocity fluid (dark color) and the sudden thickening of the boundary layer indicate the recirculation area.

We ran the computation on 12,000 cores of the Hazel Hen system at the HLRS, corresponding to a load of 9800 DOFs/core. It took 47,000 CPUh to advance the simulation by one time unit, including all analyze and I/O routines. Thus, we achieved a PID of $1.35 \mu\text{s}$, which is larger than the values presented in Section 4.6. Part of this can be explained by additional analyze work (e.g. calculation of time-averages, time-accurate sampling of the solution at specific points), but the larger part of the increase is due to the additional work introduced by the split formulation of the volume integral.

6.2. Shock-vortex interaction

This example shows how the shock-capturing mechanism based on FV subcells can be used to simulate flows with shocks, while retaining high order accuracy in smooth regions. We simulate a shock-vortex interaction on a domain of size $[0, 2] \times [0, 1]$, where a vertical, stationary shock of Mach number $Ma = 1.5$ is initialized at $x = 0.5$ based on the Rankine–Hugoniot condition. To the left of the shock, a traveling, isotropic vortex is superimposed on the flow field, see [93] for details of the initial solution. We do not take viscous effects into account, so the Euler equations are considered for this simulation. The two-dimensional mesh consists of 200×100 cells, and we run a fifth-order simulation. The left and right sides are connected through periodic boundary conditions, while the top and bottom of the domain are treated as walls. In the DG cells, the split DG formulation is used, and we employ entropy conserving flux formulations after Chandrashekar [94]. To detect the shock region, an indicator function origination from the Jameson–Schmidt–Turkel scheme [74] is used. It is based on pointwise comparisons of the pressure with neighboring maximal and minimum pressure values.

Fig. 21 shows results from the simulation at two different time instances. On the left side, the density is visualized, while the right side shows the distribution of FV and DG cells. The top row corresponds to a time where the isotropic vortex is traveling directly through the shock, significantly distorting the formerly straight shock front. Nonetheless, the indicator function detects the location of the front, and we are able to only thread the immediate vicinity of the shock with the FV scheme. No artificial waves are originating from the shock front, which would be typical for the behavior of high order schemes when high gradients are present. At the time instance depicted in the bottom row, the vortex has traveled further to the right, and the shock front is again nearly vertical. Due to the interaction with the shock, the vortex is severely distorted. However, it is still clearly visible. At the chosen resolution, this is only possible because the high order DG method was used after the shock, and it is able to convect the vortex further downstream without excessive artificial damping.

We have shown here, that the proposed hybrid DG/FV scheme can capture shocks and prevent them from introducing spurious oscillations, while the high order method used in the smooth regions of the flow is able to leverage its low dissipation and dispersion errors.

6.3. Aeroacoustic cavity noise

The third example demonstrates the applicability of *FLEXI* to perform direct noise computations (DNC). As an example, we consider wall-bounded turbulent flow over cavities. Industrial applications can be found in the context

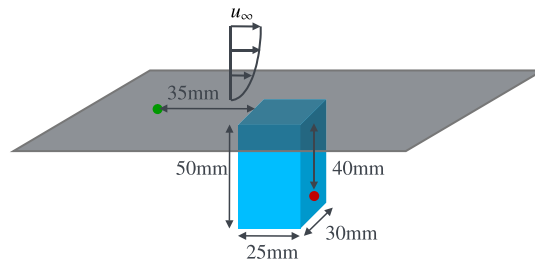


Fig. 22. Schematic sketch of the open cavity configuration from [98] including the acoustic sensor positions inside the cavity and in front (● Microphone, ● Kulite sensor).

of an automobile exterior such as sunroof buffeting as well as noise emitted from small gaps between exterior panels, e.g. the door gap. Here, different noise generation mechanisms have been identified such as Rossiter feedback, Helmholtz resonance and standing waves. All physical mechanisms have their source of noise in pressure fluctuations of the incoming turbulent boundary layer above the cavity opening. To capture the large bandwidth of scales between acoustics and turbulence, a DNC of those effects results in a challenging multiscale problem which requires high order numerics. DNC is necessary to resolve the two-way interaction between hydrodynamics and acoustics. Therefore, both requirements, high order numerics as well as DNC, make *FLEXI* a promising compressible flow solver to resolve aeroacoustic noise.

To perform cavity simulations with an upstream turbulent boundary layer, a zonal LES approach is indispensable, since simulating laminar to turbulent transition in addition could exceed computational resources. Therefore, *FLEXI* has been augmented by a new turbulent inflow condition combining weak anisotropic linear forcing by de Meux [51] and a recycling rescaling approach similar to Lund [95], in order to get a defined turbulent inflow which reveals good properties in terms of artificial noise emission. To reduce computational cost further, the wall-model by Kawai and Larsson [96] is used in combination with Vreman's eddy-viscosity model [46] to account for small-scale dissipation.

Based on the geometric definition of a benchmark experiment, which is described in detail in [97], DNC simulations of the open rectangular cavity given in Fig. 22 have been performed. This specific setup includes Rossiter-feedback as well as excited standing waves with a non-linear interaction between both. The inflow velocity is chosen as $U_\infty = 42.5$ m/s with a corresponding momentum thickness Reynolds number at the leading edge of $Re_\theta = 6300$ at standard conditions. The computational setup has 30 Mio. DOFs per solution variable, a polynomial degree of $N = 7$ is used and the equally spaced y^+ approximation at the wall is $y^+ = 15$. The computation was run for $\Delta t = 0.2$ s on HLRS Cray XC40 HPC system at a computational cost of 80,000 CPUh. To validate the accuracy of our framework, results of the wall pressure signal measured inside the cavity and in front of the cavity (see Fig. 22) are compared to the results of the benchmark experiment. Fig. 23 (left) shows the Fourier transformed pressure signal inside the cavity. Above $f = 2$ kHz the spectrum is characterized by various peaks that all correspond to different acoustic room modes. Here, both pressure level as well as the frequency width of the peaks are in good accordance with the experimental results, especially when comparing the same time averaging intervals. Below $f = 2$ kHz the spectrum reveals the first two Rossiter feedback modes as well as the first quarter wave. The underlying interaction between hydrodynamics and acoustics plays an essential role to capture the given phenomena. This demonstrates that *FLEXI* is capable to depict defined multiscale physics accurately. Fig. 23 (right) compares the wall pressure spectrum in front of the cavity. Below $f = 2$ kHz experiment and numerics agree well. Especially, Vreman's eddy-viscosity model helps to reduce artificial pressure fluctuations caused by the wall-model. Above $f = 2$ kHz the measured pressure signal drops significantly, which reveals limits of the pressure transducer used in the experiment, especially when comparing to Hu's semi-empirical wall pressure model. In summary, *FLEXI* is capable to resolve multiscale problems such as the interaction of hydrodynamics and acoustics within cavity flows, which cannot be captured by state-of-the-art hybrid approaches. Further, we have demonstrated that in case of cavity noise, a great amount of turbulence modeling can be introduced to significantly reduce computational cost while still preserving accuracy of the acoustic results.

7. Conclusion and outlook

High order schemes offer a number of potential benefits for the numerical simulation of multiscale problems. However, leveraging their advantages also requires algorithms for computation, pre- and post-processing which are specifically designed with HO in mind. Up to now, commercially available software lacks stringent and consistent HO support. *FLEXI* provides an open-source toolchain for HO fluid dynamics simulations, combining the *HOPR* mesh generator, the DG-based solver for the compressible Navier–Stokes equations itself and the post-processing suite *POSTI* into a seamless framework. We have presented *FLEXI*, its underlying numerical algorithms, features and capabilities as well as implementation details and aspects important to usability and reproducibility in this work here. In order to keep the discussion concise and to the point, we have focused on aspects that are (a) already available as open-source and (b) that are directly relevant to single-phase, compressible turbulent flows. There are however a number of ongoing extension to *FLEXI* towards a full multi-physics framework with a strong focus on applicability to complex engineering problems which have not been

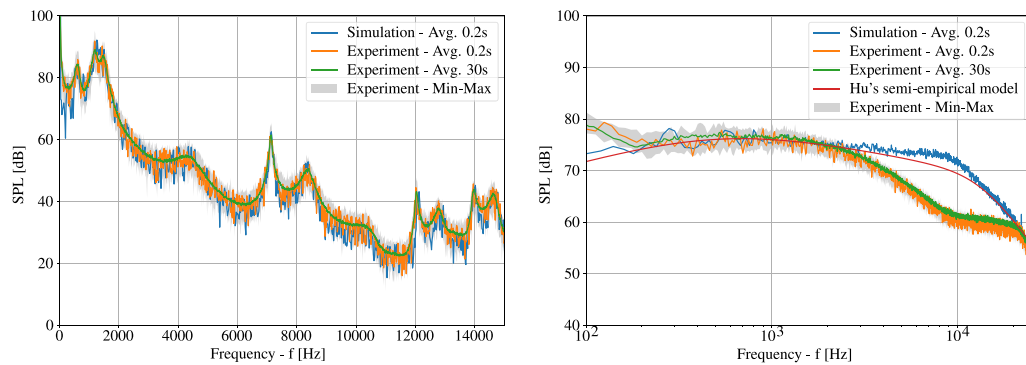


Fig. 23. (left) Comparison of the pressure spectrum measured inside the cavity. (right) Comparison of the wall pressure spectrum measured in front of the cavity.

mentioned here. Among them are:

- Multiphase and multicomponent capabilities, in which phase boundaries are tracked with a sharp interface approach [99]
- Complex equations of state based on realistic models or tabulated data [100]
- A Lagrangian particle tracking method for high order geometry, used in LES and DNS of particle laden flows [101]
- Asymptotic consistent low Mach number schemes based on IMEX splittings [34]
- Semi-implicit and fully implicit time integration schemes [35]
- Mesh deformation and mesh moving based on ALE formulations [102]
- An overset/Chimera mesh module
- Sliding mesh interface for stator/rotor flows
- A coupled particle-in-cell and direct simulation Monte Carlo solver for reactive plasma flows [103,104]
- Intrusive and non-intrusive methods for uncertainty quantification of the Navier–Stokes equation [105]
- A flexible and modular framework for the creation and management of simulation stacks and automatic, optimal scheduling on HPC systems
- A data-exchange interface to OpenFoam⁴ and a coupling to the preCICE library [106] for coupled multiphysics simulations

We plan on incorporate most of these features into the open-source version of *FLEXI* in order to provide a mature and feature-rich HO framework to the community.

CRediT authorship contribution statement

Nico Kraiss: Methodology, Software, Investigation, Writing - original draft, Writing - review & editing, Visualization. **Andrea Beck:** Conceptualization, Methodology, Software, Supervision, Writing - original draft, Writing - review & editing, Visualization. **Thomas Bolemann:** Methodology, Software. **Hannes Frank:** Methodology, Software. **David Flad:** Methodology, Software. **Gregor Gassner:** Conceptualization, Methodology, Supervision. **Florian Hindenlang:** Conceptualization, Methodology, Software, Investigation, Writing - original draft. **Malte Hoffmann:** Methodology, Software. **Thomas Kuhn:** Methodology, Software, Investigation, Writing - original draft. **Matthias Sonntag:** Methodology, Software, Investigation. **Claus-Dieter Munz:** Conceptualization, Supervision.

Acknowledgments

The authors gratefully acknowledge the support and computing resources granted by the High Performance Computing Center Stuttgart (HLRS) on the national supercomputer Cray XC40 *Hazel Hen* under the grants *hpcdg* and *SEAL*. *FLEXI* has been developed, tested and applied by a number of people, both from the Numerics Research Group at the University of Stuttgart and elsewhere. The authors acknowledge their contributions and work. A possibly incomplete list of contributors can be found in the *FLEXI* Doxygen:

<https://www.flexi-project.org/doc/doxygen/html/>

⁴ <https://openfoam.org/>

The authors would also like to thank Andrei Cimpoero from CFMS Services Ltd., for his valuable input by using and testing *HOPR*, for providing the mesh of the serrated nozzle and for organizing the webinar on high order CFD technologies, where *HOPR* was presented. Gregor Gassner has been supported by the European Research Council (ERC) under the European Union's Eights Framework Program Horizon 2020 with the research project Extreme, ERC Grant Agreement No. 714487.

References

- [1] I. Silberman, Planetary waves in the atmosphere, *J. Meteorol.* 11 (1) (1954) 27–34.
- [2] S.A. Orszag, Numerical methods for the simulation of turbulence, *Phys. Fluids* 12 (12) (1969) II–250.
- [3] R. Dahlburg, J. Picone, Pseudospectral simulation of compressible magnetohydrodynamic turbulence, *Comput. Methods Appl. Mech. Engrg.* 80 (1–3) (1990) 409–416.
- [4] M. Yokokawa, K. Itakura, A. Uno, T. Ishihara, Y. Kaneda, 16.4-Tflops direct numerical simulation of turbulence by a Fourier spectral method on the Earth Simulator, in: SC'02: Proceedings of the 2002 ACM/IEEE Conference on Supercomputing, IEEE, 2002, p. 50.
- [5] M. Dumbser, M. Käser, Arbitrary high order non-oscillatory finite volume schemes on unstructured meshes for linear hyperbolic systems, *J. Comput. Phys.* 221 (2) (2007) 693–723.
- [6] C.-W. Shu, High-order finite difference and finite volume WENO schemes and discontinuous Galerkin methods for CFD, *Int. J. Comput. Fluid Dyn.* 17 (2) (2003) 107–118.
- [7] M. Svärd, M.H. Carpenter, J. Nordström, A stable high-order finite difference scheme for the compressible Navier–Stokes equations, far-field boundary conditions, *J. Comput. Phys.* 225 (1) (2007) 1020–1038.
- [8] Y. Maday, A.T. Patera, Spectral element methods for the incompressible Navier–Stokes equations, in: *State-of-the-Art Surveys on Computational Mechanics*, 1989, pp. 71–143.
- [9] F. Bassi, S. Rebay, A high-order accurate discontinuous finite element method for the numerical solution of the compressible Navier–Stokes equations, *J. Comput. Phys.* 131 (2) (1997) 267–279.
- [10] B. Cockburn, G.E. Karniadakis, C.-W. Shu, *Discontinuous Galerkin Methods: Theory, Computation and Applications*, Vol. 11, Springer Science & Business Media, 2012.
- [11] C.D. Cantwell, D. Moxey, A. Comerford, A. Bolis, G. Rocco, G. Mengaldo, D. De Grazia, S. Yakovlev, J.-E. Lombard, D. Ekelschot, B. Jordi, H. Xu, Y. Mohamied, C. Eskilsson, B. Nelson, P. Vos, C. Biotto, R.M. Kirby, S.J. Sherwin, Nektar plus plus : An open-source spectral/hp element framework, *Comput. Phys. Comm.* 192 (2015) 205–219.
- [12] H. Klimach, K. Jain, S. Roller, End-to-end parallel simulations with APES, in: *PARCO*, 2013, pp. 703–711.
- [13] F.D. Witherden, A.M. Farrington, P.E. Vincent, PyFR: An open source framework for solving advection–diffusion type problems on streaming architectures using the flux reconstruction approach, *Comput. Phys. Comm.* 185 (11) (2014) 3028–3040.
- [14] K. Hillewaert, Development of the Discontinuous Galerkin Method for High-Resolution, Large Scale CFD and Acoustics in Industrial Geometries, Presses univ. de Louvain, 2013.
- [15] G. Gassner, D.A. Kopriva, A comparison of the dispersion and dissipation errors of Gauss and Gauss–Lobatto discontinuous Galerkin spectral element methods, *SIAM J. Sci. Comput.* 33 (5) (2011) 2560–2579.
- [16] R.C. Moura, G. Mengaldo, J. Peiró, S.J. Sherwin, On the eddy-resolving capability of high-order discontinuous Galerkin approaches to implicit LES/under-resolved DNS of Euler turbulence, *J. Comput. Phys.* 330 (2017) 615–623.
- [17] D. Drikakis, M. Hahn, A. Mosedale, B. Thornber, Large eddy simulation using high-resolution and high-order methods, *Phil. Trans. R. Soc. A* 367 (1899) (2009) 2985–2997.
- [18] F. Bassi, S. Rebay, High-order accurate discontinuous finite element solution of the 2D Euler equations, *J. Comput. Phys.* 138 (2) (1997) 251–285, <http://dx.doi.org/10.1006/jcph.1997.5454>.
- [19] M.J. Brazell, D.J. Mavriplis, High-order discontinuous Galerkin mesh resolved turbulent flow simulations of a NACA 0012 airfoil, in: 53rd AIAA Aerospace Sciences Meeting, 2015, p. 1529.
- [20] A. Uranga, P.-O. Persson, M. Drela, J. Peraire, Implicit large eddy simulation of transitional flows over airfoils and wings, in: 19th AIAA Computational Fluid Dynamics, 2009, p. 4131.
- [21] L.T. Diosady, S.M. Murman, DNS of flows over periodic hills using a discontinuous Galerkin spectral-element method, in: 44th AIAA Fluid Dynamics Conference, 2014, p. 2784.
- [22] J.-E.W. Lombard, D. Moxey, S.J. Sherwin, J.F. Hoessler, S. Dhandapani, M.J. Taylor, Implicit large-eddy simulation of a wingtip vortex, *AIAA J.* 54 (2) (2016) 506–518.
- [23] M. Bohm, A.R. Winters, G.J. Gassner, D. Derigs, F. Hindenlang, J. Saur, An entropy stable nodal discontinuous Galerkin method for the resistive MHD equations. Part I: Theory and numerical verification, *J. Comput. Phys.* (2018) <http://dx.doi.org/10.1016/j.jcp.2018.06.027>.
- [24] F. Hindenlang, Mesh Curving Techniques for High Order Parallel Simulations on Unstructured Meshes (Ph.D. thesis), University of Stuttgart, 2014, <http://dx.doi.org/10.18419/opus-3957>.
- [25] F. Hindenlang, T. Bolemann, C.-D. Munz, Mesh curving techniques for high order discontinuous Galerkin simulations, in: *IDIHOM: Industrialization of High-Order Methods-a Top-Down Approach*, Springer, 2015, pp. 133–152.
- [26] C. Geuzaine, J.-F.c. Remacle, Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities, *Internat. J. Numer. Methods Engrg.* 79 (11) (2009) 1309–1331, <http://dx.doi.org/10.1002/nme.2579>.
- [27] T. Toulorge, C. Geuzaine, J.-F. Remacle, J. Lambrechts, Robust untangling of curvilinear meshes, *J. Comput. Phys.* 254 (2013) 8–26, <http://dx.doi.org/10.1016/j.jcp.2013.07.022>.
- [28] M. Fortunato, P.-O. Persson, High-order unstructured curved mesh generation using the Winslow equations, *J. Comput. Phys.* 307 (2016) 1–14, <http://dx.doi.org/10.1016/j.jcp.2015.11.020>.
- [29] J. Marcon, D.A. Kopriva, S.J. Sherwin, J. Peiró, A high resolution PDE approach to quadrilateral mesh generation, *J. Comput. Phys.* (2019) <http://dx.doi.org/10.1016/j.jcp.2019.108918>.
- [30] The HDF Group, Hierarchical data format, version 5, 1997–2019, <http://www.hdfgroup.org/HDF5/>.
- [31] G. Farin, *Curves and Surfaces For CAGD: A Practical Guide*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2002.
- [32] G. Farin, D. Hansford, Discrete Coons patches, *Comput. Aided Geom. Design* 16 (7) (1999) 691–700, [http://dx.doi.org/10.1016/S0167-8396\(99\)00031-X](http://dx.doi.org/10.1016/S0167-8396(99)00031-X).
- [33] A. de Boer, A. van Zuijlen, H. Bijl, Radial basis functions for interface interpolation and mesh deformation, in: *Advanced Computational Methods in Science and Engineering*, Springer, 2009, pp. 143–178.
- [34] J. Zeifang, K. Kaiser, A. Beck, J. Schütz, C.-D. Munz, Efficient high-order discontinuous Galerkin computations of low mach number flows, *Commun. Appl. Math. Comput. Sci.* 13 (2) (2018) 243–270.
- [35] S. Vangelatos, On the Efficiency of Implicit Discontinuous Galerkin Spectral Element Methods for the Unsteady Compressible Navier–Stokes Equations (Ph.D. thesis), University of Stuttgart, 2019.

- [36] M. Atak, J. Larsson, C.-D. Munz, The multicore challenge: Petascale DNS of a spatially-developing supersonic turbulent boundary layer up to high Reynolds numbers using DGSEM, in: M.M. Resch, W. Bez, E. Focht, H. Kobayashi, J. Qi, S. Roller (Eds.), *Sustained Simulation Performance 2015*, Springer International Publishing, Cham, 2015, pp. 171–183.
- [37] C.W. Schulz-Rinne, Classification of the Riemann problem for two-dimensional gas dynamics, *SIAM J. Math. Anal.* 24 (1) (1993) 76–88.
- [38] G.J. Gassner, A.R. Winters, D.A. Kopriva, Split form nodal discontinuous Galerkin schemes with summation-by-parts property for the compressible Euler equations, *J. Comput. Phys.* 327 (2016) 39–66.
- [39] A.D. Beck, D.G. Flad, C. Tönhäuser, G. Gassner, C.-D. Munz, On the influence of polynomial de-aliasing on subgrid scale models, *Flow Turbul. Combust.* 97 (2) (2016) 475–511, <http://dx.doi.org/10.1007/s10494-016-9704-y>.
- [40] D. Flad, G. Gassner, On the use of kinetic energy preserving DG-schemes for large eddy simulation, *J. Comput. Phys.* 350 (2017) 782–795.
- [41] A.D. Beck, T. Bolemann, D. Flad, H. Frank, G.J. Gassner, F. Hindenlang, C.-D. Munz, High-order discontinuous Galerkin spectral element methods for transitional and turbulent flow simulations, *Internat. J. Numer. Methods Fluids* 76 (8) (2014) 522–548.
- [42] E.F. Toro, *Riemann Solvers and Numerical Methods for Fluid Dynamics: a Practical Introduction*, Springer Science & Business Media, 2013.
- [43] F. Bassi, S. Rebay, G. Mariotti, S. Pedinotti, M. Savini, A high-order accurate discontinuous finite element method for inviscid and viscous turbomachinery flows, in: R. Decuyper, G. Dibelius (Eds.), *2nd European Conference on Turbomachinery Fluid Dynamics and Thermodynamics*, 1997, pp. 99–108.
- [44] J. Smagorinsky, General circulation experiments with the primitive equations: I. The basic experiment, *Mon. Weather Rev.* 91 (3) (1963) 99–164.
- [45] F. Nicoud, H.B. Toda, O. Cabrit, S. Bose, J. Lee, Using singular values to build a subgrid-scale model for large eddy simulations, *Phys. Fluids* 23 (8) (2011) 085106.
- [46] A. Vreman, An eddy-viscosity subgrid-scale model for turbulent shear flow: Algebraic theory and applications, *Phys. Fluids* 16 (10) (2004) 3670–3681.
- [47] D. Flad, A. Beck, C.-D. Munz, Simulation of underresolved turbulent flows by adaptive filtering using the high order discontinuous Galerkin spectral element method, *J. Comput. Phys.* 313 (2016) 1–12.
- [48] J. Larsson, S. Kawai, J. Bodart, I. Bermejo-Moreno, Large eddy simulation with modeled wall-stress: recent progress and future directions, *Mech. Eng. Rev.* 3 (1) (2016) 15–00418.
- [49] P. Spalart, S. Allmaras, A one-equation turbulence model for aerodynamic flows, in: *30th Aerospace Sciences Meeting and Exhibit*, 1992, p. 439.
- [50] S. Collis, Discontinuous Galerkin methods for turbulence simulation, in: *Proceedings of the Summer Program 2002*, Center for Turbulence Research, 2002, pp. 155–167.
- [51] B. De Laage de Meux, B. Audebert, R. Manceau, R. Perrin, Anisotropic linear forcing for synthetic turbulence generation in large eddy simulation and hybrid RANS/LES modeling, *Phys. Fluids* 27 (3) (2015) 035115.
- [52] C. Pruett, T. Gatski, C.E. Grosch, W. Thacker, The temporally filtered Navier–Stokes equations: properties of the residual stress, *Phys. Fluids* 15 (8) (2003) 2127–2140.
- [53] J. Ahrens, B. Geveci, C. Law, *Paraview: An end-user tool for large data visualization*, in: *The Visualization Handbook*, Vol. 717, Elsevier München, 2005.
- [54] A. Beck, N. Krais, Blender pipeline, 2018, <https://github.com/flexi-framework/BlenderPipeline>.
- [55] Blender Online Community, Blender - a 3D Modelling and Rendering Package, Blender Foundation, Stichting Blender Foundation, Amsterdam, 2018, <http://www.blender.org>.
- [56] D.A. Kopriva, Metric identities and the discontinuous spectral element method on curvilinear meshes, *J. Sci. Comput.* 26 (3) (2006) 301.
- [57] D.A. Kopriva, G. Gassner, On the quadrature and weak form choices in collocation type discontinuous Galerkin spectral element methods, *J. Sci. Comput.* 44 (2) (2010) 136–155.
- [58] D.A. Kopriva, A conservative staggered-grid Chebyshev multidomain method for compressible flows. II. a semi-structured method, *J. Comput. Phys.* 128 (2) (1996) 475–488.
- [59] T.C. Fisher, M.H. Carpenter, High-order entropy stable finite difference schemes for nonlinear conservation laws: Finite domains, *J. Comput. Phys.* 252 (2013) 518–557.
- [60] E. Tadmor, The numerical viscosity of entropy stable schemes for systems of conservation laws. I, *Math. Comp.* 49 (179) (1987) 91–103.
- [61] A. Jameson, Formulation of kinetic energy preserving conservative schemes for gas dynamics and direct numerical simulation of one-dimensional viscous compressible flow in a shock tube using entropy and kinetic energy preserving schemes, *J. Sci. Comput.* 34 (2) (2008) 188–208.
- [62] G. Blaisdell, E. Spyropoulos, J. Qin, The effect of the formulation of nonlinear terms on aliasing errors in spectral methods, *Appl. Numer. Math.* 21 (3) (1996) 207–219.
- [63] C.A. Kennedy, A. Gruber, Reduced aliasing formulations of the convective terms within the Navier–Stokes equations for a compressible fluid, *J. Comput. Phys.* 227 (3) (2008) 1676–1700.
- [64] J. von Neumann, R.D. Richtmyer, A method for the numerical calculation of hydrodynamic shocks, *J. Appl. Phys.* 21 (3) (1950) 232–237.
- [65] P.-O. Persson, J. Peraire, Sub-cell shock capturing for discontinuous Galerkin methods, in: *44th AIAA Aerospace Sciences Meeting and Exhibit*, 2006, p. 112.
- [66] J. Qiu, C.-W. Shu, Hermite WENO schemes and their application as limiters for Runge–Kutta discontinuous Galerkin method: one-dimensional case, *J. Comput. Phys.* 193 (1) (2004) 115–135.
- [67] A. Huerta, E. Casoni, J. Peraire, A simple shock-capturing technique for high-order discontinuous Galerkin methods, *Internat. J. Numer. Methods Fluids* 69 (10) (2012) 1614–1632.
- [68] M. Sonntag, C.-D. Munz, Efficient parallelization of a shock capturing for discontinuous Galerkin methods using finite volume sub-cells, *J. Sci. Comput.* 70 (3) (2017) 1262–1289.
- [69] M. Sonntag, C.-D. Munz, Shock capturing for discontinuous Galerkin methods using finite volume subcells, in: *Finite Volumes for Complex Applications VII–Elliptic*, in: *Parabolic and Hyperbolic Problems*, Springer, 2014, pp. 945–953.
- [70] A. Harten, High resolution schemes for hyperbolic conservation laws, *J. Comput. Phys.* 49 (3) (1983) 357–393.
- [71] B. Van Leer, Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov's method, *J. Comput. Phys.* 32 (1) (1979) 101–136.
- [72] M. Sonntag, *Shape Derivatives and Shock Capturing for the Navier–Stokes Equations in Discontinuous Galerkin Methods* (Ph.D. thesis), University of Stuttgart, 2017.
- [73] M. Dumbser, O. Zanotti, R. Loubère, S. Diot, A posteriori subcell limiting of the discontinuous Galerkin finite element method for hyperbolic conservation laws, *J. Comput. Phys.* 278 (2014) 47–75.
- [74] A. Jameson, W. Schmidt, E. Turkel, Numerical solution of the Euler equations by finite volume methods using Runge Kutta time stepping schemes, in: *14th Fluid and Plasma Dynamics Conference*, 1981, p. 1259.
- [75] F. Ducros, V. Ferrand, F. Nicoud, C. Weber, D. Darraq, C. Gacherieu, T. Poinso, Large-eddy simulation of the shock/turbulence interaction, *J. Comput. Phys.* 152 (2) (1999) 517–549.

- [76] J. Williamson, Low-storage Runge-Kutta schemes, *J. Comput. Phys.* 35 (1) (1980) 48–56.
- [77] R. Courant, K. Friedrichs, H. Lewy, Über die partiellen Differenzengleichungen der mathematischen Physik, *Math. Ann.* 100 (1) (1928) 32–74.
- [78] B. Cockburn, C.-W. Shu, Runge–Kutta discontinuous Galerkin methods for convection-dominated problems, *J. Sci. Comput.* 16 (3) (2001) 173–261.
- [79] T. Warburton, T. Hagstrom, Taming the CFL number for discontinuous Galerkin methods on structured meshes, *SIAM J. Numer. Anal.* 46 (6) (2008) 3151–3180.
- [80] C. Rumsey, B. Wedan, T. Hauser, M. Poinot, Recent updates to the CFD general notation system (CGNS), in: 50th AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition, 2012, p. 1264.
- [81] N. Fehn, W.A. Wall, M. Kronbichler, A matrix-free high-order discontinuous Galerkin compressible Navier–Stokes solver: A performance comparison of compressible and incompressible formulations for turbulent incompressible flows, *Internat. J. Numer. Methods Fluids* 89 (3) (2019) 71–102.
- [82] T. Bolemann, Towards Industrialization of High-Order Discontinuous Galerkin Methods for Turbulent Flows (Ph.D. thesis), University of Stuttgart, 2018.
- [83] M. Resch, A. Kaminski, The epistemic importance of technology in computer simulation and machine learning, *Minds Mach.* 29 (1) (2019) 9–17, <http://dx.doi.org/10.1007/s11023-019-09496-5>.
- [84] R.D. Peng, Reproducible research in computational science, *Science* 334 (6060) (2011) 1226–1227.
- [85] S. Chacon, B. Straub, *Pro Git*, Apress, 2014.
- [86] G.J. Gassner, A.D. Beck, On the accuracy of high-order discretizations for underresolved turbulence simulations, *Theor. Comput. Fluid Dyn.* 27 (3–4) (2013) 221–237.
- [87] F. Hindenlang, G.J. Gassner, C. Altmann, A. Beck, M. Staudenmaier, C.-D. Munz, Explicit discontinuous Galerkin methods for unsteady problems, *Comput. & Fluids* 61 (2012) 86–93.
- [88] D. Flad, A.D. Beck, G. Gassner, C.-D. Munz, A discontinuous Galerkin spectral element method for the direct numerical simulation of aeroacoustics, in: 20th AIAA/CEAS Aeroacoustics Conference, 2014, p. 2740.
- [89] M. Atak, A. Beck, T. Bolemann, D. Flad, H. Frank, C.-D. Munz, High fidelity scale-resolving computational fluid dynamics using the high order discontinuous Galerkin spectral element method, in: *High Performance Computing in Science and Engineering’ 15*, Springer, 2016, pp. 511–530.
- [90] H.M. Frank, C.-D. Munz, Direct aeroacoustic simulation of acoustic feedback phenomena on a side-view mirror, *J. Sound Vib.* 371 (2016) 132–149.
- [91] S. Pirozzoli, Numerical methods for high-speed flows, *Annu. Rev. Fluid Mech.* 43 (2011) 163–194.
- [92] M. Drela, XFOIL: An analysis and design system for low Reynolds number airfoils, in: T.J. Mueller (Ed.), *Low Reynolds Number Aerodynamics*, Springer Berlin Heidelberg, Berlin, Heidelberg, 1989, pp. 1–12.
- [93] C.-W. Shu, Essentially non-oscillatory and weighted essentially non-oscillatory schemes for hyperbolic conservation laws, in: *Advanced Numerical Approximation of Nonlinear Hyperbolic Equations*, Springer, 1998, pp. 325–432.
- [94] P. Chandrashekar, Kinetic energy preserving and entropy stable finite volume schemes for compressible Euler and Navier–Stokes equations, *Commun. Comput. Phys.* 14 (5) (2013) 1252–1286.
- [95] T.S. Lund, X. Wu, K.D. Squires, Generation of turbulent inflow data for spatially-developing boundary layer simulations, *J. Comput. Phys.* 140 (2) (1998) 233–258.
- [96] S. Kawai, J. Larsson, Wall-modeling in large eddy simulation: Length scales, grid resolution, and accuracy, *Phys. Fluids* 24 (1) (2012) 015105.
- [97] L. Erbig, N. Hu, S. Lardeau, Experimental and numerical study of passive gap noise, in: 2018 AIAA/CEAS Aeroacoustics Conference, 2018, p. 3595.
- [98] T. Kuhn, Zonal large eddy simulation of active open cavity noise using a high order discontinuous Galerkin method, in: 25th AIAA/CEAS Aeroacoustics Conference, 2019, p. 2465.
- [99] S. Fechter, C.-D. Munz, C. Rohde, C. Zeiler, A sharp interface method for compressible liquid–vapor flow with phase transition and surface tension, *J. Comput. Phys.* 336 (2017) 347–374.
- [100] F. Hempert, S. Boblest, T. Ertl, F. Sadlo, P. Offenhäuser, C. Glass, M. Hoffmann, A. Beck, C.-D. Munz, U. Iben, Simulation of real gas effects in supersonic methane jets using a tabulated equation of state with a discontinuous Galerkin spectral element method, *Comput. & Fluids* 145 (2017) 167–179.
- [101] A. Beck, P. Ortwein, P. Kopper, N. Krais, D. Kempf, C. Koch, Towards high-fidelity erosion prediction: On time-accurate particle tracking in turbomachinery, *Int. J. Heat Fluid Flow* 79 (2019) 108457.
- [102] G. Schnücke, N. Krais, T. Bolemann, G.J. Gassner, Entropy stable discontinuous Galerkin schemes on moving meshes for hyperbolic conservation laws, *J. Sci. Comput.* (2020) <http://dx.doi.org/10.1007/s10915-020-01171-7>.
- [103] S.M. Copplestone, M. Pfeiffer, S. Fasoulas, C.-D. Munz, High-order Particle-In-Cell simulations of laser-plasma interaction, *Eur. Phys. J. Spec. Top.* 227 (14) (2019) 1603–1614.
- [104] P. Ortwein, S. Copplestone, C.-D. Munz, T. Binder, A. Mirza, P. Nizikov, M. Pfeiffer, W. Reschke, S. Fasoulas, Piclas: A highly flexible particle code for the simulation of reactive plasma flows, in: 2017 IEEE International Conference on Plasma Science, ICOPS, IEEE, 2017, p. 1.
- [105] T. Kuhn, J. Dürrwächter, F. Meyer, A. Beck, C. Rohde, C.-D. Munz, Uncertainty quantification for direct aeroacoustic simulations of cavity flows, *J. Comput. Acoust.* (2018).
- [106] H.-J. Bungartz, F. Lindner, B. Gatzhammer, M. Mehl, K. Scheufele, A. Shukaev, B. Uekermann, PreCICE – A fully parallel library for multi-physics surface coupling, in: *Advances in Fluid-Structure Interaction*, *Comput. & Fluids* 141 (2016) 250–258, <http://dx.doi.org/10.1016/j.compfluid.2016.04.003>.